ABWA Campus Management System — Core + Student Attendance Specification

Nov 3, 2026 · @Usman Munir

1. Overview

Phase 1 delivers one multi-campus core for ABWA Knowledge Village plus a Student Attendance module, launched first at ABWA Medical College. Every later module (timetable, exams, fees, HR, LMS) plugs into the same core instead of becoming a separate app.

What Phase 1 must do

- Host many campuses under one ABWA Knowledge Village account: Medical College, Nursing College, Paramedical College, University, and any added later.
- Keep each campus's faculty, students, departments and data fully separate, while letting a Village-level Super Admin see across all of them.
- Let a College Administrator create classes, split a class into sections, assign teachers, and enrol students.
- Let teachers log every class held (date, time, topic) and mark each student's attendance, section by section.
- Calculate attendance percentages, flag students below the required threshold, and export reports.

Out of scope for Phase 1 (designed for, not built): timetable auto-generation, student and parent portals, leave applications, SMS/WhatsApp alerts, exams, fees, faculty attendance, hostel, transport, library.

Guiding principles

1. **One core, many campuses.** Every record carries `campusId`; nothing is hard-coded to the Medical College.
2. **Every class has at least one section.** An undivided class gets a single default section called "All". This keeps attendance logic identical for divided and undivided classes.
3. **Attendance is a log of sessions.** A session is one class meeting for one section; attendance records hang off sessions.
4. **Configuration over code.** Thresholds, status codes, lock windows and term types are campus settings, because a medical college (annual years) and a university (semesters) run differently.

5. **Nothing is silently changed.** Submitted attendance locks; edits need a reason and are written to an audit log.
6. **Mobile-first for teachers.** A teacher must be able to mark 150 students on a phone in under a minute.

Technology choice

The system is built in Google AI Studio Build mode on its native PostgreSQL integration: a React front end, a Node.js server, Cloud SQL for PostgreSQL as the database, and Firebase Authentication for sign-in. AI Studio now provisions all three from the chat, and publishes to Cloud Run (Google Cloud docs; AI Studio Build docs).

Layer	Choice	Why it fits ABWA
Front end	React + TypeScript + Tailwind (AI Studio default)	Mobile-first teacher screens and dense admin tables from one codebase
Server	Node.js runtime inside the AI Studio app (Express-style API)	All business rules, permission checks and percentage calculations run on the server, never in the browser
Database	Cloud SQL for PostgreSQL 18, created by AI Studio's "Enable database" card	Relational data (campus → class → section → session → student) needs foreign keys, unique constraints and transactions; SQL makes reports simple
ORM / schema	The schema file AI Studio generates (a TypeScript schema, e.g. Drizzle) plus versioned migrations	Every schema change is a reviewed migration, not an ad-hoc edit
Sign-in	Firebase Authentication: Google sign-in and email/password	Teachers without Google accounts can still log in; roles live in PostgreSQL, not in Firebase
Hosting	Cloud Run, via AI Studio's Publish button	One-click deploy; scales with use

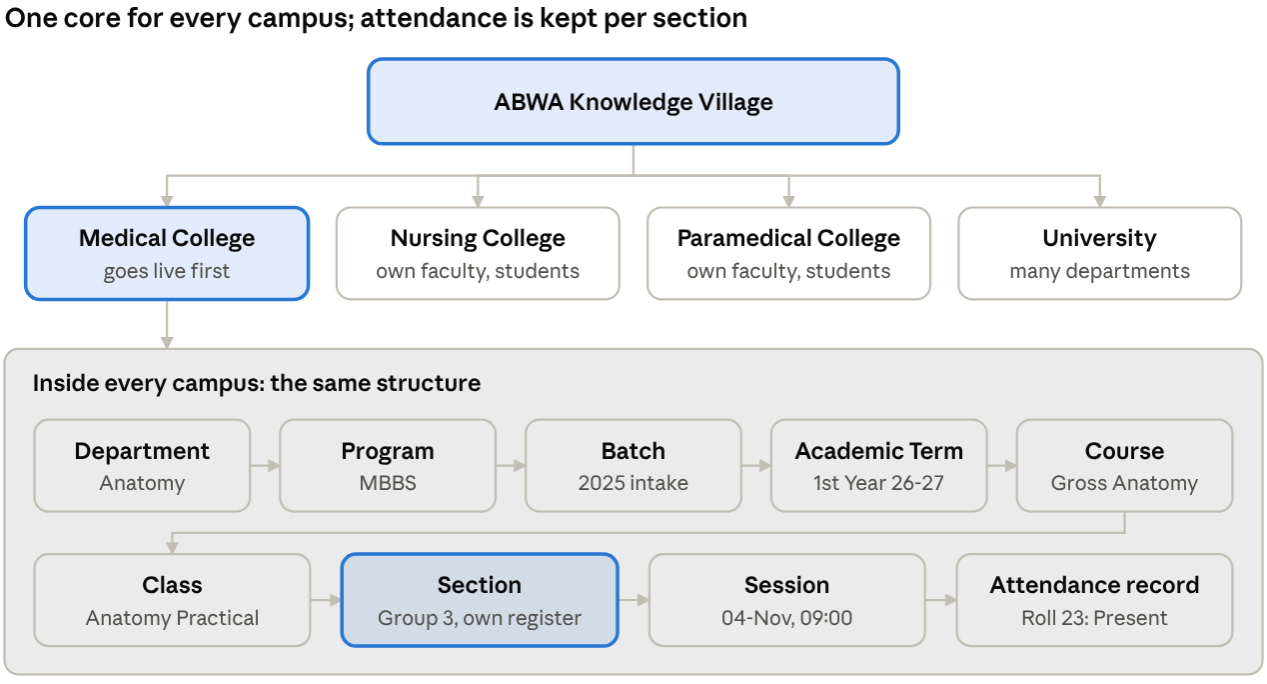
Limits to know before starting

- On the free Starter Tier, the database is a developer-edition instance with 0.5 vCPU, 2 GB RAM and **1 GB storage**, and you can build only **2 apps** with Cloud SQL. That is enough to build and pilot one college.

- Developer edition has **no automated backups, no high availability and no replicas**. Before real attendance goes in, link a billing-enabled Google Cloud project and plan backups (section 9).
- With a billing project the instance can grow to 2 vCPU, 8 GB RAM and 20 GB storage. Beyond that, the database can be migrated to a standard Cloud SQL instance later; the app code does not change.
- **The region you pick in the "Enable database" card cannot be changed later.** Pick the closest available region to Pakistan.
- Google Workspace accounts are not eligible for the free Starter Tier. If ABWA uses Workspace, build with a personal Google account or an ABWA Cloud project with billing.
- Organisations with domain-restricted sharing policies may block the integration; check with whoever manages ABWA's Google Cloud.

2. Organisational hierarchy and glossary

The system nests eleven levels, from ABWA Knowledge Village down to a single attendance mark. Universities use departments heavily; colleges still get departments (Anatomy, Physiology, and so on) so the model is the same everywhere.



ABWA CMS hierarchy · campus to attendance record

Every campus repeats the same structure below it; the Section is where each register is kept separately.

Level	What it is	Medical College example
Organisation	The single top tenant	ABWA Knowledge Village
Campus	An institution under the Village, with its own admin, faculty and students	ABWA Medical College (code AMC)
Department	A teaching department inside a campus	Anatomy, Physiology, Biochemistry
Program	A degree or qualification	MBBS
Batch	An intake cohort moving through the program together	MBBS Batch 2025 (Session 2025–2030)
Academic Term	A year or semester the batch is currently in	1st Year MBBS 2025–26
Course	A subject or module in the curriculum	Gross Anatomy; Foundation Module
Class	A course taught to a batch in a term, of one type, with assigned teachers	1st Year Anatomy — Lecture
Section	A sub-group of a class with its own roster and its own attendance	Section A (Roll 1–50), Practical Group 3
Session	One meeting of one section on a date, logged by a teacher	Section A, 04-Nov-2026, 09:00–10:00, "Brachial plexus"
Attendance record	One student's status in one session	Roll 23: Present

Key rules about these entities

- A **class** always has a type: Lecture, Tutorial, Practical/Lab, Small Group Discussion (SGD), Clinical Rotation/Ward, Skills Lab, Demonstration. The same course usually has several classes, e.g. Anatomy Lecture (whole batch, one section) and Anatomy Practical (four groups).
- A **student belongs to exactly one section per class**, but can be in different sections for different classes (Section A for lectures, Group 3 for practicals).
- **Teachers are assigned at class level** (they can mark any section) **or section level** (only that section). Several teachers per class is normal in a medical college.
- A **term type** is a campus setting: *Annual* (Medical, Nursing) or *Semester* (University). Terms have start and end dates and one is marked current per batch.

- **IDs** follow editable patterns per campus, e.g. students `AMC-MBBS-25-001`, faculty `AMC-F-0042`.

2.1 Medical college terms used in this document

In everyday speech at the college, "class" usually means a whole year ("the 1st Year class"); in this system that is a **Year class** (a batch in its current year), and **Class** always means one subject taught to that year in one teaching format.

Term	Meaning in the system	Example
Year class	A batch in its current academic year	1st Year MBBS (Batch 2026)
Class (subject class)	One discipline × one teaching format for a year class	1st Year — Anatomy — Practical
Group set	A way of dividing a year class into groups, reused by many classes	"Teaching groups A–F", "Clinical groups 1–8"
Group	One part of a group set; a student is in exactly one group per set	Group C (Roll 61–90)
Section	The attendance unit of a class; usually linked to a group so its roster follows the group	Anatomy Practical — Group C
Teaching slot	One time period where different groups are taught different subjects at the same time	Mon 09:00–11:00: A Anatomy, B Physiology, C Biochemistry
Block	A UHS curriculum block, numbered across the programme (three per year in the 2023 modular curriculum)	Block 1 (Year 1)
Module	A system-based unit inside a block	Foundation-I; Hematopoietic & Lymphatic
Hospital (clinical) department	A teaching department based in ABWA Hospital, with wards, OPDs and theatres	Medicine, Surgery, Gynaecology & Obstetrics
Clinical rotation	A group's posting to a hospital department for a date range	Group 3 in Surgery, 06-Jan to 31-Jan

Term	Meaning in the system	Example
Clinical supervisor	Consultant or registrar who teaches and marks attendance during a rotation	Senior Registrar, Surgery Unit II

UHS's Modular Integrated Curriculum 2K23 lists Year 1 as Block-1 (Foundation-I; Hematopoietic & Lymphatic), Block-2 (Musculoskeletal & Locomotion-I) and Block-3 (UHS curriculum, Year 1). Block and module names are entered as data, so ABWA can follow whichever UHS version applies to each batch.

3. Users, roles and permissions

Access is role-based and scoped: a role is granted at a level (Organisation, Campus or Department), and applies only inside that scope. One person can hold several role assignments, e.g. a professor who teaches at the Medical College and the Nursing College.

Role	Scope	Can do
Super Admin	Organisation	Create/archive campuses; appoint Campus Admins; set Village-wide defaults; view every campus's reports; read audit logs everywhere
Campus Admin (College Administrator)	One campus	Manage departments, programs, batches, terms, courses, classes, sections; add/import faculty and students; assign teachers; enrol students; edit locked attendance with reason; approve correction requests; all campus reports; campus settings
Principal / Viewer	One campus	Read-only dashboards and reports for the campus

Role	Scope	Can do
Head of Department (HOD)	One department	View classes and reports of the department; approve correction requests for department classes (optional, per campus setting). Cannot create classes, assign teachers, sections, groups, rotations or substitutes
Teacher (Faculty)	Assigned classes/sections	See own classes; create sessions; mark and submit attendance; edit before lock; request corrections after lock; view own class reports
Student	Self	Phase 2: view own attendance per class, percentage and shortage warnings
Parent	Linked student	Future: view child's attendance, receive alerts

Permission rules

- Every read and write is checked against the user's role assignments **and** the record's `campusId` (and `departmentId` where relevant). A Medical College admin can never see Nursing College data.
- A teacher can mark attendance only for sections they are assigned to (directly, or through the parent class). A Campus Admin can assign a **substitute teacher** to a single session.
- Only Super Admin and Campus Admin can create users. Users sign in with email/password or Google sign-in; accounts are disabled, never deleted, so history stays intact.
- Every role change is written to the audit log.

Action-by-role matrix

Y = allowed within scope, Own = only own assigned classes/sections or own record, R = read-only, — = not allowed. The server enforces every cell; the UI only hides what the server would refuse. **Department Heads are view-and-approve roles: they never create or assign classes, teachers, sections, groups, rotations or substitutes.**

Action	Super Admin	Campus Admin	Principal	HOD	Teacher / Clinical supervisor	Student (Ph. 2)
Create / archive campus	Y	—	—	—	—	—
Edit campus settings (threshold, lock window)	Y	Y	—	—	—	—
Create users, assign roles	Y	Y (own campus, not Super Admin)	—	—	—	—
Departments, programs, batches, terms, calendar	Y	Y	R	R (own dept)	—	—
Select or change a department head	Y	Y	—	—	—	—
Curriculum blocks and modules	Y	Y	R	R	R	—
Courses (disciplines)	Y	Y	R	R (own dept)	R (own)	—
Register students (draft, verify, activate)	Y	Y	R	—	—	—
Register teachers and staff	Y	Y	R	R (own dept)	Own profile (R)	—
View CNIC, phone, address, documents	Y	Y	—	—	—	Own
Create classes, assign teachers	Y	Y	—	—	—	—
Create group sets and groups, move students between groups	Y	Y	—	—	—	—
Create / split sections, link sections to groups	Y	Y	—	—	—	—
Create parallel teaching slots	Y	Y	—	—	—	—
Plan clinical rotations, assign supervisors	Y	Y	—	R (own dept)	R (own rotations)	—
Enrol / un-enrol students	Y	Y	—	—	—	—
Assign substitute for a session	Y	Y	—	—	—	—

Action	Super Admin	Campus Admin	Principal	HOD	Teacher / Clinical supervisor	Student (Ph. 2)
Create session, mark attendance	Y	Y	—	—	Own	—
Edit submitted session before lock	Y	Y	—	—	Own	—
Edit locked session (reason required)	Y	Y	—	—	—	—
Cancel session (reason required)	Y	Y	—	—	Own (before lock)	—
Raise correction request	—	—	—	—	Own	—
Approve / reject correction	Y	Y	—	Y if enabled (own dept)	—	—
Campus dashboard and all reports	Y	Y	R	R (own dept)	Own classes	Own card
Export PDF / Excel	Y	Y	Y	Y (own dept)	Own classes	Own card
View audit log	Y	Y (own campus)	—	—	Own sessions' trail	—
Recalculate attendance summaries	Y	Y	—	—	—	—

A department head who also teaches holds the Teacher role as well, and marks attendance for their own assigned sections like any teacher.

4. Core module: functional specification

The core holds everything other modules depend on: campuses, academic structure, people, classes, sections and enrolment.

4.1 Campus management (Super Admin)

- Create a campus: name, short code (AMC, ANC, APC, AU), type (College / University), term type (Annual / Semester), address, logo, contact, status (Active / Archived).
- Per-campus settings: attendance threshold %, warning band %, lock window (hours), late rule, excused-leave rule, ID patterns, working days, time zone (default Asia/Karachi), date format (DD-MMM-YYYY).
- Appoint one or more Campus Admins.
- Campus switcher in the header for anyone with roles in more than one campus.

4.2 Academic structure (Campus Admin)

- **Departments:** name, code, HOD (optional), status.
- **Programs:** name (MBBS), code, duration in years, owning department (optional), term type inherited from campus.
- **Batches:** program, intake year, session label (2025–2030), current term, status (Active / Graduated / Archived).
- **Academic terms:** name (1st Year 2025–26 or Fall 2026), start date, end date, type, linked batches.
- **Courses:** code, title, department, program, year/semester, credit hours (optional), component types offered (Lecture, Practical…).
- **Academic calendar:** holidays and exam blocks per campus; marking attendance on a holiday shows a warning.

4.3 Departments and department heads (Campus Admin)

Every department has a type, a location (college or hospital) and an optional head selected from its own staff; hospital departments also list the wards, OPDs and theatres where students are taught.

Department fields: code, name, type, location, parent department (for units such as Surgery Unit I / Unit II), head of department, contact extension, status.

Type	Location	ABWA Medical College examples (editable seed list)
Basic science	College	Anatomy, Physiology, Biochemistry
Para-clinical	College	Pathology, Pharmacology, Forensic Medicine, Community Medicine, Behavioural Sciences
Clinical — Medicine and allied	Hospital	Medicine, Cardiology, Pulmonology, Gastroenterology, Nephrology, Neurology, Psychiatry, Dermatology
Clinical — Surgery and allied	Hospital	General Surgery, Orthopaedics, Neurosurgery, Urology, Plastic Surgery, Paediatric Surgery, Anaesthesia
Clinical — other	Hospital	Gynaecology & Obstetrics, Paediatrics, ENT, Ophthalmology, Radiology, Emergency Medicine
Education and support	College	Medical Education (DME), Examination, Library, IT, Administration, Accounts

Hospital teaching locations (for hospital departments): name, kind (ward, OPD, OT, ER, ICU, lab, skills lab, lecture room), department, floor/block, capacity. Example: "Surgical Ward 2 — Surgery Unit II".

Selecting a department head

1. On the department page the Campus Admin clicks **Select head**, picks from staff whose primary or secondary department is this one, enters the start date and (optionally) the appointment order number.
2. The system closes the previous head's term (end date = day before), writes a `department_heads` history row, sets the department's current head, grants the new head the **HOD** role scoped to this department and removes it from the previous head.
3. A department can have no head (shown as "Vacant" and flagged in the Setup checklist). One person can head more than one department.

What a department head can and cannot do

- Can: view the department's staff, classes, sections, groups, rotations, sessions and attendance reports; approve or reject correction requests for the department's classes when the campus setting allows; export department reports.
- Cannot: create or edit classes, assign or change teachers, create sections or groups, move students, plan rotations, assign substitutes, register students or staff. Those stay with the Campus Admin.

4.4 Student Registration Module (Campus Admin, Admissions office)

Student Registration is built and used before attendance: a student exists in the system only after their record is entered, documents verified and the record activated, and only active students appear on any attendance roster.

Registration states

1. **Draft** — admissions staff are filling the form; can be saved incomplete.
2. **Submitted for verification** — all required fields filled; documents uploaded or marked "seen in original".
3. **Verified** — Campus Admin has checked the documents; or **Returned** with a note of what is missing.
4. **Active** — registration number and roll number issued, batch and year class set, groups assigned; the student now appears on rosters.
5. Later statuses: Freeze, Struck off, Left, Graduated (with effective date and reason, kept in history).

Form sections and fields (* = required for activation)

Section	Fields
Admission	Program*, batch / session*, admission date*, admission category* (configurable: Open Merit, Self-Finance, Overseas Pakistani, Foreign, other), merit number, MDCAT roll number and score, aggregate %, admission letter number, fee challan / voucher number (reference only)
Identity	Full name as on Matric certificate*, father's name*, gender*, date of birth*, CNIC or B-Form number*, nationality*, passport number (foreign students), domicile province* and district*, blood group (for ID card and emergencies), photo* (passport size, white background)
Contact	Student mobile*, WhatsApp number, personal email*, permanent address*, current / postal address, city*
Guardians	Father / guardian name*, relation*, CNIC, occupation, mobile*, email; second guardian; emergency contact name, relation and mobile* (can be a guardian)
Previous education	Matric / O-Level: board or system, year, roll number, obtained and total marks*. FSc (Pre-Medical) / A-Level: board or system, year, roll number, obtained and total marks*. IBCC equivalence number and date for O/A-Level students
University registration	UHS registration (enrolment) number and date, once issued by the affiliating university
Residence	Hostel resident (yes/no), hostel and room (filled by the future Hostel module), day scholar transport route (future Transport module)
Documents	Matric certificate, FSc / A-Level certificate, IBCC equivalence, MDCAT result, CNIC / B-Form, domicile, father's / guardian's CNIC, photographs, character certificate, medical fitness certificate, undertaking / affidavit, admission letter. Each: uploaded file, "original seen" tick, verified by, verified date

Section	Fields
Allocation (on activation)	Registration number* (from campus pattern, e.g. AMC-MBBS-26-001), roll number*, year class*, teaching group, clinical group (later years), status

Rules and validation

- CNIC/B-Form: 13 digits, shown as 00000-0000000-0; unique per campus. Mobile: Pakistani format (03XX-XXXXXXX) or international with country code.
- Duplicate check on CNIC, MDCAT roll number and name + father's name + date of birth, with a warning listing the matching record.
- Date of birth must make the student at least 16 at admission; marks cannot exceed totals.
- Registration and roll numbers are issued only on activation, never reused, even if a student leaves.
- CNIC, phone, address, guardians and documents are visible to Campus Admin and Super Admin only.
- Bulk path: Excel import creates students in **Draft** or directly **Active** (admin's choice), using the template in section 11.2; documents can be attached afterwards.

Outputs: printable registration form (A4 with photo), student profile page, list of students with filters, document-pending report, and later an ID card print (Phase 2).

4.5 Teachers and Staff Registration Module (Campus Admin, HR / Admin office)

One staff register holds every employee of a campus, teaching or not, each with a category, a designation and a primary department; only staff in a teaching or clinical category can be assigned to classes and mark attendance.

Staff categories and designations (designations are a configurable list with a rank order, so reports sort by seniority)

Category	Can teach / mark attendance	Designations (editable seed list)
Teaching faculty (basic and para-clinical)	Yes	Professor, Associate Professor, Assistant Professor, Senior Lecturer, Lecturer, Senior Demonstrator, Demonstrator, Visiting Faculty

Category	Can teach / mark attendance	Designations (editable seed list)
Clinical faculty (hospital)	Yes	Professor, Associate Professor, Assistant Professor, Senior Registrar, Registrar, Consultant, Medical Officer, House Officer (supervised)
Academic support	Optional (e.g. skills lab sessions)	Medical Educationist, Coordinator, Skills Lab Instructor, Librarian
Technical staff	No	Lab Technologist, Lab Technician, Lab Assistant, Lab Attendant, IT Officer
Administrative staff	No	Principal's Secretary, Admin Officer, Accounts Officer, Clerk, Computer Operator, Office Assistant

Form sections and fields (* = required)

Section	Fields
Identity	Full name*, father's / husband's name*, gender*, date of birth*, CNIC*, nationality, domicile, photo*
Contact	Mobile*, official email*, personal email, permanent and current address, emergency contact name, relation and mobile*
Employment	Staff code* (campus pattern, e.g. AMC-S-0042), category*, designation*, primary department*, secondary departments (e.g. a Physiology lecturer who also teaches in a module), unit (for hospital departments), employment type* (permanent, contract, visiting, part-time, adjunct), BPS / grade, joining date*, contract end date, reporting to, status (active, on leave, resigned, retired, terminated) with effective date

Section	Fields
Professional (teaching and clinical)	PMDC registration number and expiry date (warning 60 days before expiry), qualifications list (degree e.g. MBBS, FCPS, MS, MD, MPhil, PhD; specialty; institution; year), teaching experience in years, specialties / subjects able to teach, publications count (optional)
Documents	CNIC copy, degrees, PMDC certificate, experience letters, appointment letter, joining report; each with verified-by and date
System access	Create login (sends invite to official email); roles granted: Teacher automatically for teaching and clinical categories; others only if the admin grants them

Rules

- CNIC unique per campus; staff code issued on save and never reused.
- A person working in two campuses has one login and one staff record per campus.
- Only Active staff in a teaching or clinical category appear in teacher pickers, section teacher lists, rotation supervisor lists and substitute pickers.
- When a staff member leaves, their login is disabled, their open class and section assignments are listed for reassignment, and their past sessions keep their name.
- Department head selection (4.3) only offers staff of that department.

Outputs: staff directory by department and designation, staff profile, PMDC expiry report, department strength report (designation counts per department), teacher workload (assigned classes, sections and sessions taught).

4.6 Classes (Campus Admin)

- Create a class: course, batch, term, class type, title (auto-suggested e.g. "1st Year — Anatomy — Lecture"), default duration, venue (optional), status.
- Assign one or more teachers; mark one as Lead.
- On creation, the system creates a default section "All" and enrols every active student of the batch into it (option to skip).
- Copy classes from the previous term to speed up a new academic year.

4.7 Sections (Campus Admin)

- Split a class into sections: name (A, B, Group 1…), capacity (optional), assigned teachers, venue.
- **Auto-split helpers:** by roll-number ranges (1–50, 51–100), into N equal groups, alphabetical, or copy section membership from another class.
- Manual drag-and-drop or checkbox moves between sections.
- Rule: a student is in exactly one section of a class. Moving a student keeps their past records in the old section and starts new ones in the new section.
- A section with existing sessions can be archived but not deleted.

4.8 Enrolment (Campus Admin)

- Enrol by batch (all active students), by selection, or by CSV of registration numbers.
- Repeaters and supplementary students can be enrolled into a class of a different batch.
- Enrolment has effective dates: attendance percentage counts only sessions held while the student was enrolled.

4.9 UHS curriculum Blocks and Modules (tags)

Blocks and modules are tags with dates, so every session is automatically labelled with the block and module running on that day, and attendance can be reported by discipline, by module or by block.

- **Curriculum version:** e.g. "UHS Modular Integrated Curriculum 2K23"; each batch is linked to one version, so older batches can stay on an older curriculum.
- **Blocks:** number (1, 2, 3 … numbered across the programme), name, year of study, and for each batch a start and end date plus the block exam dates.
- **Modules:** code, name, block, order, planned weeks, and for each batch a start and end date. Example for Year 1 (2K23): Block-1 → Foundation-I, Hematopoietic & Lymphatic; Block-2 → Musculoskeletal & Locomotion-I; Block-3 → as listed in the UHS document.
- **Longitudinal / spiral subjects** (e.g. Behavioural Sciences, Islamiyat, Pakistan Studies, research) can be tagged "spiral" so they run across modules without breaking the tags.
- **Auto-tagging:** when a session is created the system fills Block and Module from the batch's block/module calendar for that date. The teacher can change the tag (e.g. a revision lecture from the previous module), and the change is logged.
- **Classes stay per discipline** (Anatomy, Physiology, Biochemistry…), because teaching departments, teachers and registers are per discipline; the block and module tags cut across them.

- **Reports gain two filters and two levels:** attendance by module and by block, per student and per group, alongside per class and per course.

4.10 Student groups and parallel group teaching (MBBS)

A year class is divided into groups once, and those groups are reused by every subject, so at the same hour Group A can be in an Anatomy practical, Group B in a Physiology practical and Group C in a Biochemistry SGD, each with its own teacher and its own attendance.

- **Group sets:** a Campus Admin creates one or more group sets per year class, e.g. "Teaching groups A–F" (for practicals, tutorials, SGDs) and later "Clinical groups 1–8" (for hospital rotations). Each set has groups with a name, code and colour.
- **Filling groups:** by roll-number ranges, into N equal groups, alphabetically, by importing an Excel list, or by hand. Every active student of the year class must be in exactly one group of each set; the Setup checklist shows anyone unassigned.
- **Linking classes to groups:** in the Section Manager, "Split by group set" creates one section per group (e.g. Anatomy Practical — Group A … Group F), each **linked** to its group. A linked section's roster follows the group automatically: when a student moves from Group B to Group D on a date, their enrolment in every linked section moves on that date, and past marks stay where they were.
- **Whole-class teaching:** lectures for the full year class keep a single section "All"; a class can have both kinds (e.g. Anatomy Lecture = All, Anatomy Practical = 6 groups).
- **Teaching slots:** a Campus Admin creates a slot (date, start and end time, year class) and assigns each group a class, teacher and venue. Saving the slot creates one planned session per group, so every teacher sees their group in My Classes at that time.
- **Clash checks:** a student cannot be booked in two sessions at the same time (checked across all their sections); a teacher cannot be in two places at once; a venue cannot hold two groups at once. Clashes are refused with a clear message showing the clashing session.
- **Recurring slots:** a slot can repeat weekly within a module's dates, with a rotation option (A→B→C) so groups move through subjects week by week.

4.11 Clinical departments and rotations (hospital teaching)

Students in clinical years go to ABWA Hospital departments in groups for fixed periods; a rotation plan says which group is in which department when, and the clinical supervisors there mark each group's attendance in the ward, OPD or theatre.

- **Clinical classes:** for each clinical year class the Campus Admin creates one clinical class per hospital department (e.g. "4th Year — Surgery — Clinical"), with clinical sub-types: ward round, bedside teaching, OPD, OT, emergency duty, case presentation /

CPC, skills lab.

- **Rotation plan:** a grid of clinical groups × date ranges × departments (e.g. Group 3: Medicine 04-Nov to 01-Dec, Surgery 02-Dec to 29-Dec). Saving the plan enrols each group into that department's clinical class (section per group) with enrolment dates equal to the rotation dates. Clashes (one group in two departments at once) are refused.
- **Supervisors:** each rotation names a lead supervisor and other clinical staff (Teacher role, clinical category) who may mark that group; units (Surgery Unit I / II) can be assigned separately.
- **Location:** clinical sessions record the hospital teaching location (ward, OPD, OT…) chosen from the department's list; the location appears on registers and reports.
- **Rotation board:** a weekly view of where every clinical group is, for the Campus Admin, Principal and department heads (read-only for heads).
- **Rotation completion:** at the end of a rotation the system shows each student's attendance for that posting against the clinical threshold (a separate campus setting, default the same as the general threshold), so short postings can be flagged for repeat.

5. Student Attendance module: functional specification

Attendance is recorded per session per section, then rolled up to per-student, per-class percentages that drive shortage warnings and reports.

5.1 Attendance statuses

Code	Status	Counts as	Notes
P	Present	Attended	Default when "Mark all present" is used
A	Absent	Not attended	
L	Late	Attended (default)	Campus setting: count as present, or every N lates = 1 absence
E	Excused / Leave	Excluded from total (default)	Campus setting: exclude, or count as present. Requires a reason
M	Medical leave	Same as Excused	Separate code so reports can show it

Status codes and labels are campus-configurable; the counting rule of each code is what the calculations read.

5.2 Session (class log)

Every class meeting is a session. Fields: section, class, date, start time, end time, duration (auto), session type (inherited from class, overridable), topic covered (required), teacher who conducted it, substitute flag, venue, remarks.

Session states:

1. **Draft** — created, attendance being marked, not counted in reports.
2. **Submitted** — counted in reports; teacher can still edit until the lock window ends (default 24 hours after session end).
3. **Locked** — read-only for the teacher; changes only via correction request or by a Campus Admin with a reason.
4. **Cancelled** — class did not happen (reason required); excluded from all percentages.

Duplicate guard: the same section cannot have two sessions overlapping in time on the same date.

5.3 Marking attendance (teacher)

- Teacher opens **My Classes**, picks a section, taps **Take attendance**. Date and time default to now; topic field is required.
- Roster shows roll number, name and photo, sorted by roll number, with a search box.
- **Mark all present** then tap absentees, or **Mark all absent** then tap those present. Tap cycles P → A → L → E.
- Live counter: Present / Absent / Late / Excused / Total.
- **Save draft** (works offline and syncs later) or **Submit**. Submit shows a confirmation with the counts.
- Back-dated sessions are allowed within a campus-set limit (default 7 days); older ones need Campus Admin.

5.4 Corrections

- After lock, a teacher raises a **correction request**: which students, old status, new status, reason.
- Campus Admin (or HOD, if enabled) approves or rejects with a comment. Approved changes apply and are audited.
- Campus Admin can edit directly, but a reason is mandatory and the change is audited.

5.5 Calculation rules

```
\text{Attendance \%} = \frac{P + L_{counted}}{\text{Total held} - E_{excluded} - M_{excluded}} \times 100
```

- **Total held** = submitted or locked sessions of the student's section, held while the student was enrolled, excluding cancelled sessions.
- Percentages are kept at three levels: per class (e.g. Anatomy Lecture), per course (all classes of Anatomy combined, optionally weighted by session hours), and overall for the term.
- **Threshold:** campus setting, default 75%. **Warning band:** default 75–80%. Below threshold = Shortage; inside band = At risk. Confirm the exact figure against current PMDC/affiliating university rules before go-live.
- Calculation can be by **session count** or **contact hours** (campus setting) since practicals and lectures differ in length.

5.6 Reports and dashboards

Report	Who	Filters	Export
Daily campus summary: sessions held, present %, sections not yet marked	Campus Admin, Principal	Date, department, batch	PDF, Excel
Section register: monthly grid of students × sessions with P/A/L/E	Teacher, Admin	Class, section, month	PDF (print-ready register), Excel
Student attendance card: every class, sessions held/attended, %	Admin, Teacher, (Student later)	Student, term	PDF
Shortage list: students below threshold or in warning band	Admin, HOD	Batch, course, threshold	PDF, Excel
Class log / teaching record: sessions held, topics covered, by whom	Admin, HOD	Class, teacher, date range	PDF, Excel
Teacher activity: sessions conducted, unmarked or late-submitted sessions	Admin	Teacher, date range	Excel
Correction log: all changes after lock	Admin, Super Admin	Date range, class	Excel

Report	Who	Filters	Export
Village overview: attendance % by campus	Super Admin	Date range	PDF

5.7 Worked calculation example

The same student can be OK, At risk or in Shortage depending on three campus settings, so those settings must be decided before go-live (section 14).

Student Roll 23, 1st Year MBBS, mid-term:

- **Anatomy Practical, Group 3** (2-hour sessions): 40 sessions created, 2 cancelled → 38 held. Marked P 30, L 3, A 3, M 2.
- **Anatomy Lecture, Section All** (1-hour sessions): 50 held. P 34, A 16.

Setting combination	Practical %	Lecture %	Anatomy (course) %
Late = present, Medical excluded, by sessions	(30+3) / (38−2) = 91.7% OK	34 / 50 = 68.0% Shortage	(33+34) / (36+50) = 77.9% At risk
3 lates = 1 absence, Medical excluded, by sessions	(30+3−1) / 36 = 88.9% OK	68.0% Shortage	(32+34) / 86 = 76.7% At risk
Late = present, Medical counted present, by sessions	(33+2) / 38 = 92.1% OK	68.0% Shortage	(35+34) / 88 = 78.4% At risk
Late = present, Medical excluded, by contact hours	66 h / 72 h = 91.7% OK	34 h / 50 h = 68.0% Shortage	100 h / 122 h = 82.0% OK

The same student is in Shortage for Anatomy Lecture, At risk for Anatomy by session count, and OK for Anatomy by contact hours. The system shows all three levels; the campus decides which one blocks exam eligibility.

5.8 Edge cases and business rules

Situation	Rule
Student admitted mid-term	Enrolment `effective_from` = admission date; only later sessions count
Student moved Group 1 → Group 2	Old enrolment closed, new one opened; old marks stay in Group 1's register; the class % combines both periods

Situation	Rule
Student struck off, frozen or left	Not on new rosters from the status date; history kept; reinstatement opens a new enrolment
Two teachers mark the same section at once	Second save is rejected with "updated by X at 09:42, reload" (row version check); no silent overwrite
Weak network, Submit tapped twice	Each submit carries a client-generated idempotency key; the server stores the first and ignores repeats
Teacher forgot to mark	Session can be back-dated up to the campus limit (default 7 days); older needs Campus Admin
Two sections taught together in one hall	"Combined session": teacher picks several of their sections, marks one merged roster; the server writes one session per section sharing a `combined_group_id`
Clinical rotation groups change every few weeks	Sections hold enrolments with effective dates; the admin re-assigns groups on the rotation date without losing earlier marks
Repeater from an older batch	Enrolled into the current batch's class; still keeps their own batch on their profile
Leave approved after the session locked	Campus Admin changes A → E/M with reason; audit entry written; summaries recalculated
Campus-wide closure (strike, weather, exam day)	Calendar event "closure" bulk-cancels that day's sessions with one reason
Session marked on a holiday	Warning shown; allowed only with a reason
Section with zero students	Cannot start attendance; flagged in the Setup checklist
Teacher leaves ABWA	Account disabled; classes reassigned; past sessions keep `conducted_by` unchanged
Class or section with sessions	Cannot be deleted, only archived
Session dates and times	Stored as date + local time in Asia/Karachi, plus a UTC timestamp for audit

5.9 Take-attendance screen in detail

This is the most-used screen in the system and is designed for a teacher standing in a lecture hall with one hand on a phone.

1. **Header card:** class title, section name, student count, date (today, editable within back-date limit), start and end time (defaults from class duration), session type, **Topic** (required, with the last 5 topics of this class as quick picks), substitute toggle.
2. **Bulk bar:** "All present", "All absent", "Reset". Sticky at the top while scrolling.
3. **Roster:** one row per student: roll number (large), name, small photo, status chip. Tap the row to cycle P → A → L → E. Long-press opens a sheet for M (medical) and a remark. Rows sorted by roll number; a search box filters by roll or name.
4. **Grid view toggle:** compact tiles showing only roll numbers (6–8 per row on a phone) for the fastest marking of large sections.
5. **Live counters** in a sticky footer: P 112 · A 8 · L 3 · E 1 · Total 124, plus Save draft and Submit buttons.
6. **Autosave:** every change is kept on the device; if the page reloads or the network drops, the draft is restored and sent when the connection returns.
7. **Submit confirmation:** shows the counts and lists absentees by roll number so the teacher can catch a wrong tap.
8. **After submit:** read view with an Edit button and a countdown ("Editable for 23 h 12 min"); after lock, the button becomes Request correction.
9. **Desktop shortcuts:** arrow keys move between students; P, A, L, E keys set status; Ctrl+Enter submits.

5.10 Attendance for groups, parallel sessions and hospital rotations

Each group's session is its own register: six groups taught six subjects in the same hour produce six sessions, six rosters and six sets of marks, and no student can appear in two of them.

Situation	How attendance works
Lecture to the whole year class	One session on section "All"; roster = every active student of the year class
Parallel teaching slot (A Anatomy, B Physiology, C Biochemistry at 09:00)	The slot creates one planned session per group; each teacher opens their own group from My Classes and marks it; the Campus Admin sees the slot as one row with a status per group (not started / draft / submitted)

Situation	How attendance works
Teacher marks a group that was not planned	Allowed (ad-hoc session) if the teacher is assigned to that group's section, subject to the student clash check
Student in a clash	Refused at session creation with the clashing session shown; the admin fixes the slot or the group membership
Group membership changed mid-module	Linked sections update on the change date; earlier sessions keep the old roster snapshot
Hospital ward round / bedside teaching	Clinical supervisor opens My Rotations, picks today's group, ward and sub-type, marks the group; location saved on the session
Group split across two units in one department	Rotation can split a group into sub-groups per unit, each a linked section with its own supervisor
Emergency duty or evening OPD (final year)	Session with sub-type "emergency duty"; may be counted separately (campus setting: count in clinical %, or report only)
Supervisor not on campus system yet	Campus Admin marks on their behalf from the paper sheet, with "entered from paper" flag and the sheet photo attached

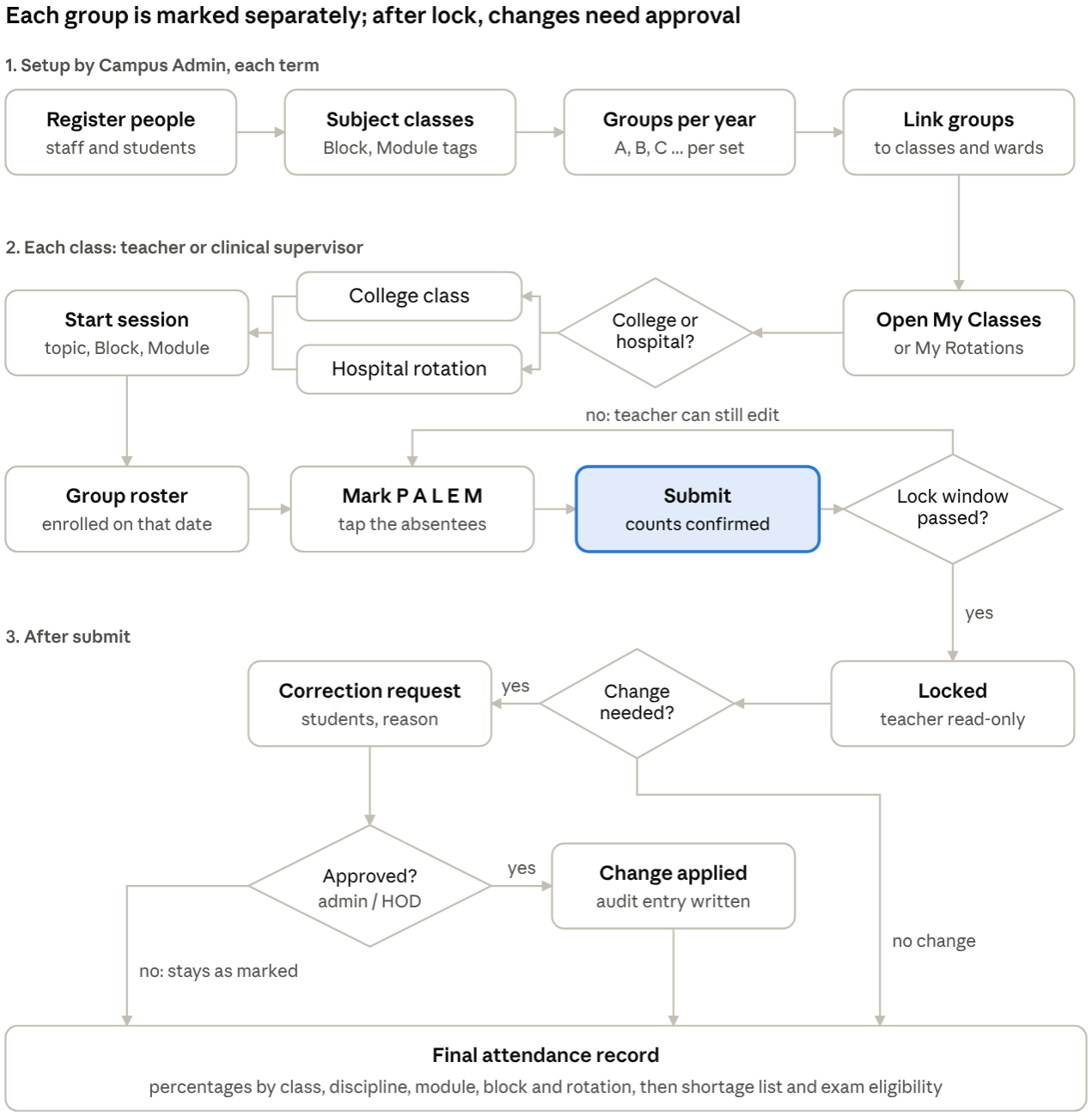
Percentages for medical students are shown at six levels: per class (e.g. Anatomy Practical), per course / discipline (Anatomy), per module, per block, per clinical rotation (posting), and per year. The campus decides which level drives exam eligibility; all levels use the same `calculateAttendance` function and the same settings, with an optional separate clinical threshold.

Extra campus settings: clinical_threshold_pct (default = threshold_pct), count_emergency_duty (yes / report only), student_clash_check (always on), allow_adhoc_group_sessions (yes / admin only).

6. End-to-end workflows

Six workflows cover Phase 1; the first runs once per campus, the second runs every day.

Attendance flowchart (ABWA Medical College)



Medical College attendance flowchart · setup, marking, lock and corrections

Setup happens once per term; every lecture, group practical or hospital rotation session then follows the same path, and after the lock window only an approved correction can change a mark.

6.1 Initial setup of ABWA Medical College

1. Super Admin signs in; organisation **ABWA Knowledge Village** is created on first run.
2. Super Admin creates campus **ABWA Medical College (AMC)**, type College, term type Annual, threshold 75%, lock window 24 h.

3. Super Admin appoints the College Administrator as Campus Admin.
4. Campus Admin creates departments with type and location: basic science and para-clinical (college) and clinical (hospital), plus hospital teaching locations (wards, OPDs, theatres).
5. **Teachers and staff registration:** imports or enters all staff with category, designation and departments; logins created for teaching and clinical staff.
6. **Department heads:** selects a head for each department from its staff.
7. Creates program **MBBS** (5 years), batches and terms; sets up the curriculum version, **blocks and modules** with dates per batch, and the academic calendar.
8. Creates courses (disciplines) per year.
9. **Student registration:** enters or imports students, verifies documents, activates them (registration and roll numbers issued).
10. Creates **group sets** per year class (teaching groups; clinical groups for years 3–5) and fills them.
11. Creates classes (discipline × year class × format) and assigns teachers; splits practical / SGD classes by group set; keeps lectures as "All".
12. Creates **teaching slots** for parallel group teaching, and the **rotation plan** for clinical years.
13. Reviews the Setup checklist (no vacant heads, no unassigned students in groups, no class without teachers, no section without students, no block or module without dates) and goes live.

6.2 Daily teacher flow

1. Teacher signs in on phone → **My Classes for today** (sections they teach, grouped by class).
2. Taps a section → **Take attendance** → enters topic.
3. Marks all present, taps absentees and late-comers, adds any excused reason.
4. Submits → session is Submitted and counts in reports.
5. Can fix mistakes until the lock window closes; after that, raises a correction request.

6.3 Admin daily monitoring

1. Campus Admin opens the dashboard: today's sessions held, campus present %, sections still unmarked.
2. Follows up on unmarked sections (teacher activity report).
3. Reviews pending correction requests and approves/rejects.
4. Weekly: runs the Shortage list per batch and shares it with HODs and the Principal.

6.4 Correction workflow

1. Teacher opens a locked session → **Request correction** → selects students, new status, reason.
2. Request appears in the Campus Admin's (or HOD's) queue with a badge count.
3. Approver approves (change applied, audit entry written, percentages recalculated) or rejects with a comment.
4. Teacher sees the outcome on the session.

6.5 New academic year rollover

1. Campus Admin creates new terms.
2. **Promote batches**: moves each batch to its next year; final-year batch becomes Graduated. Students marked as repeaters stay back.
3. **Copy classes** from the last term for each year; reassign teachers where changed.
4. Re-runs sections (copy membership or auto-split).
5. Previous term's sessions and attendance are kept read-only.

6.6 Adding a new campus (e.g. ABWA Nursing College)

1. Super Admin creates the campus with its own code, term type and settings.
2. Appoints its Campus Admin.
3. That admin repeats 6.1 steps 4–13 for their campus. No code changes are needed; Medical College data is never visible to them.
4. For ABWA University, the same flow applies with Semester terms and more departments under each faculty/school.

6.7 Student registration workflow

1. Admissions staff create a **Draft** record from the admission list (or import the list).
2. They fill the form section by section and upload or tick documents seen in original.
3. **Submit for verification**; the record appears in the Campus Admin's verification queue.
4. Campus Admin checks documents: **Verify** or **Return** with a note (returns go back to admissions).
5. **Activate**: system issues registration and roll numbers, sets the year class, and asks for teaching group (and clinical group in later years).
6. On activation the student is enrolled automatically into every "All" section of their year class and into the sections linked to their groups.
7. UHS registration number is added later when issued.

6.8 A parallel group teaching day

1. Campus Admin (or Medical Education office acting as admin) creates the week's **teaching slots**, e.g. Monday 09:00–11:00: Group A Anatomy practical (Dr X, Dissection Hall), Group B Physiology practical (Dr Y, Physiology Lab), Group C Biochemistry SGD (Dr Z, Tutorial Room 2).
2. The system checks student, teacher and venue clashes, then creates one planned session per group, auto-tagged with the current block and module.
3. At 09:00 each teacher opens My Classes, sees only their group, confirms the topic and marks attendance.
4. The slot view shows the Campus Admin which groups are submitted and which are still unmarked.
5. Next week the slot rotates (A Physiology, B Biochemistry, C Anatomy) if rotation was chosen.

6.9 Clinical rotation workflow

1. Campus Admin creates clinical groups for the clinical year class and the **rotation plan** (group × department × dates), naming a lead supervisor per rotation.
2. On the rotation start date each group's students appear in that department's clinical section; the department head can see who is posted (read-only).
3. Each day the clinical supervisor opens **My Rotations**, picks the group, sub-type (ward round, OPD, OT…) and location, and marks attendance in the hospital.
4. Sessions lock and corrections work exactly as for college classes.
5. At the end of the rotation the **rotation completion report** lists each student's posting attendance against the clinical threshold; short students are flagged for repeat posting.
6. The plan advances: the group moves to its next department on the next date range with no manual re-enrolment.

7. Data model in PostgreSQL

The database has 46 tables in snake_case, all keyed by UUID, with `campus_id` on every campus-owned row so one query filter keeps campuses apart. Attendance is stored one row per student per session, which is what PostgreSQL handles best and keeps reports to plain SQL.

Conventions for every table: `id uuid primary key default gen_random_uuid()`, `campus_id uuid not null references campuses(id)` (except organisation-level tables), `status text`, `created_at timestamptz default now()`, `created_by uuid`,

`updated_at timestamptz`, `updated_by uuid`. No hard deletes: rows are archived. All foreign keys indexed.

7.1 Table catalogue

Group	Table	Key columns and constraints
Tenancy	organizations	name, code (unique), logo_url
Tenancy	campuses	org_id, name, code (unique per org), type (college/university), term_type (annual/semester), timezone, logo_url
Tenancy	campus_settings	campus_id (PK), threshold_pct 75, warning_pct 80, clinical_threshold_pct, lock_hours 24, backdate_days 7, late_rule, late_to_absent_n 3, excused_rule, calc_basis, hod_can_approve, count_emergency_duty, allow_adhoc_group_sessions, reg_no_pattern, roll_no_pattern, staff_code_pattern
Identity	users	firebase_uid (unique), email (unique), name, phone, photo_url, is_active, last_login_at
Identity	role_assignments	user_id, role, campus_id (null = organisation), department_id (null = whole campus); unique together
Departments	departments	code (unique per campus), name, dept_type (basic_science / para_clinical / clinical / support), location (college / hospital), parent_department_id (units), hod_staff_id
Departments	department_heads	department_id, staff_id, start_date, end_date, order_no; one open row per department
Departments	hospital_locations	department_id, name, kind (ward / opd / ot / er / icu / lab / skills_lab / room), floor, capacity
Academics	programs	department_id, name, code, duration_years
Academics	batches	program_id, intake_year, session_label, current_term_id, curriculum_version_id; unique (program_id, intake_year)
Academics	academic_terms	name, term_type, start_date, end_date

Group	Table	Key columns and constraints
Academics	batch_terms	batch_id, term_id, year_or_semester_no
Academics	calendar_events	start_date, end_date, type (holiday / exam / closure / event), title
Academics	courses	department_id, program_id, code (unique per campus), title, year_or_semester, credit_hours
Curriculum	curriculum_versions	name (e.g. UHS 2K23), affiliating_body, effective_from
Curriculum	curriculum_blocks	version_id, block_no, name, year_of_study
Curriculum	curriculum_modules	block_id, code, name, sort_order, planned_weeks, is_spiral
Curriculum	batch_curriculum_dates	batch_id, block_id, module_id (nullable), start_date, end_date, exam_date
Staff	designations	category, name, rank_order; unique (campus_id, category, name)
Staff	staff	user_id, staff_code (unique per campus), category, designation_id, primary_department_id, unit_department_id, name, father_or_husband_name, gender, dob, cnic (unique per campus), mobile, official_email, personal_email, addresses, emergency contact, employment_type, bps_grade, joining_date, contract_end_date, reports_to_staff_id, pmdc_no, pmdc_expiry, experience_years, status
Staff	staff_departments	staff_id, department_id (secondary departments)
Staff	staff_qualifications	staff_id, degree, specialty, institution, year
Staff	staff_documents	staff_id, doc_type, file_url, verified_by, verified_at

Group	Table	Key columns and constraints
Students	students	program_id, batch_id, registration_state (draft / submitted / verified / returned / active), reg_no and roll_no (set on activation, unique), name, father_name, gender, dob, cnic, nationality, passport_no, domicile_province, domicile_district, blood_group, mobile, whatsapp, email, permanent_address, current_address, city, admission_date, admission_category, merit_no, mdcat_roll_no, mdcat_score, aggregate_pct, uhs_reg_no, uhs_reg_date, hostel_resident, photo_url, verified_by, verified_at, activated_at
Students	student_guardians	student_id, name, relation, cnic, occupation, mobile, email, is_emergency_contact
Students	student_academic_records	student_id, level (matric / o_level / fsc / a_level), board, year, roll_no, obtained, total, ibcc_no
Students	student_documents	student_id, doc_type, file_url, original_seen, verified_by, verified_at
Students	student_status_history	student_id, from_status, to_status, effective_date, reason, changed_by
Groups	group_sets	batch_id, term_id, name, purpose (teaching / clinical)
Groups	student_groups	group_set_id, name, code, colour
Groups	student_group_members	group_id, student_id, effective_from, effective_to; one open membership per student per set
Teaching	classes	course_id, batch_id, term_id, department_id, class_type (incl. clinical), title, default_duration_min, venue
Teaching	class_teachers	class_id, staff_id, is_lead
Teaching	sections	class_id, name, student_group_id (nullable = not linked), capacity, is_default, venue
Teaching	section_teachers	section_id, staff_id

Group	Table	Key columns and constraints
Teaching	enrollments	class_id, section_id, student_id, effective_from, effective_to, source (manual / group_link / rotation)
Teaching	teaching_slots	batch_id, slot_date, start_time, end_time, recurrence_rule, rotate_groups, until_date
Teaching	teaching_slot_assignments	slot_id, group_id, section_id, staff_id, venue
Clinical	clinical_rotations	batch_id, group_id, department_id, class_id, section_id, start_date, end_date, lead_supervisor_staff_id
Clinical	rotation_supervisors	rotation_id, staff_id
Attendance	sessions	section_id, class_id, session_date, start_time, end_time, duration_min, session_type, clinical_subtype, topic, conducted_by (staff), is_substitute, block_id, module_id, tag_overridden, location_id, slot_assignment_id, rotation_id, entered_from_paper, state, cancel_reason, combined_group_id, version, idempotency_key
Attendance	attendance_records	session_id, student_id, status (P/A/L/E/M), remark, marked_by, marked_at; unique (session_id, student_id)
Attendance	correction_requests	session_id, requested_by, reason, state, reviewed_by, reviewed_at, review_comment
Attendance	correction_request_items	request_id, student_id, from_status, to_status
Audit	audit_logs	campus_id, actor_user_id, action, entity_type, entity_id, before, after, reason, at
Audit	import_jobs	type (students / staff / enrolments / groups), file_name, total_rows, ok_rows, error_rows, errors

7.2 SQL for the attendance core

Give this to AI Studio as the reference schema (Prompt 1). It can translate it into its own schema file and migrations.

```
CREATE EXTENSION IF NOT EXISTS btree_gist;

CREATE TABLE classes (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  campus_id uuid NOT NULL REFERENCES campuses(id),
  course_id uuid NOT NULL REFERENCES courses(id),
  batch_id uuid NOT NULL REFERENCES batches(id),
  term_id uuid NOT NULL REFERENCES academic_terms(id),
  department_id uuid REFERENCES departments(id),
  class_type text NOT NULL CHECK (class_type IN

('lecture','tutorial','practical','sgd','clinical','skills_lab','demonstratio
n')),
  title text NOT NULL,
  default_duration_min int NOT NULL DEFAULT 60 CHECK (default_duration_min
BETWEEN 15 AND 480),
  venue text,
  status text NOT NULL DEFAULT 'active' CHECK (status IN
('active','archived')),
  created_at timestamptz NOT NULL DEFAULT now(), created_by uuid,
  updated_at timestamptz, updated_by uuid
);

CREATE TABLE sections (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  campus_id uuid NOT NULL REFERENCES campuses(id),
  class_id uuid NOT NULL REFERENCES classes(id),
  name text NOT NULL,
  capacity int,
  is_default boolean NOT NULL DEFAULT false,
  venue text,
  status text NOT NULL DEFAULT 'active' CHECK (status IN
('active','archived')),
  created_at timestamptz NOT NULL DEFAULT now(), created_by uuid,
  UNIQUE (class_id, name)
);

CREATE TABLE enrollments (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  campus_id uuid NOT NULL REFERENCES campuses(id),
  class_id uuid NOT NULL REFERENCES classes(id),
```

```
  section_id uuid NOT NULL REFERENCES sections(id),
  student_id uuid NOT NULL REFERENCES students(id),
  effective_from date NOT NULL,
  effective_to date,
  CHECK (effective_to IS NULL OR effective_to >= effective_from),
  created_at timestamptz NOT NULL DEFAULT now(), created_by uuid
);
-- a student has at most one open enrolment per class
CREATE UNIQUE INDEX uq_enrollment_open ON enrollments (class_id, student_id)
  WHERE effective_to IS NULL;

CREATE TABLE sessions (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  campus_id uuid NOT NULL REFERENCES campuses(id),
  class_id uuid NOT NULL REFERENCES classes(id),
  section_id uuid NOT NULL REFERENCES sections(id),
  session_date date NOT NULL,
  start_time time NOT NULL,
  end_time time NOT NULL CHECK (end_time > start_time),
  duration_min int GENERATED ALWAYS AS
    ((EXTRACT(EPOCH FROM (end_time - start_time)) / 60)::int) STORED,
  session_type text NOT NULL,
  topic text NOT NULL CHECK (length(trim(topic)) > 0),
  conducted_by uuid NOT NULL REFERENCES staff(id),
  is_substitute boolean NOT NULL DEFAULT false,
  venue text,
  state text NOT NULL DEFAULT 'draft'
    CHECK (state IN ('draft','submitted','locked','cancelled')),
  cancel_reason text,
  CHECK (state <> 'cancelled' OR cancel_reason IS NOT NULL),
  combined_group_id uuid,
  submitted_at timestamptz,
  locked_at timestamptz,
  version int NOT NULL DEFAULT 1,
  idempotency_key text UNIQUE,
  created_at timestamptz NOT NULL DEFAULT now(), created_by uuid,
  updated_at timestamptz, updated_by uuid,
  -- no two live sessions of one section may overlap in time
  EXCLUDE USING gist (
    section_id WITH =,
    tsrange(session_date + start_time, session_date + end_time) WITH &&
  ) WHERE (state <> 'cancelled')
);

CREATE TABLE attendance_records (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
```

```
  campus_id uuid NOT NULL REFERENCES campuses(id),
  session_id uuid NOT NULL REFERENCES sessions(id),
  student_id uuid NOT NULL REFERENCES students(id),
  status char(1) NOT NULL CHECK (status IN ('P','A','L','E','M')),
  remark text,
  CHECK (status NOT IN ('E','M') OR remark IS NOT NULL),
  marked_by uuid NOT NULL REFERENCES users(id),
  marked_at timestamptz NOT NULL DEFAULT now(),
  UNIQUE (session_id, student_id)
);

CREATE INDEX ix_sessions_section_date ON sessions (campus_id, section_id,
session_date);
CREATE INDEX ix_sessions_class_state  ON sessions (campus_id, class_id,
state);
CREATE INDEX ix_records_student       ON attendance_records (campus_id,
student_id);
CREATE INDEX ix_enrollments_section   ON enrollments (section_id) WHERE
effective_to IS NULL;

-- counts per student per class; percentage is applied by one shared server
function
CREATE VIEW v_student_class_attendance AS
SELECT ar.campus_id, ar.student_id, s.class_id, c.term_id,
  COUNT(*)                                    AS held,
  COUNT(*) FILTER (WHERE ar.status = 'P')     AS present,
  COUNT(*) FILTER (WHERE ar.status = 'L')     AS late,
  COUNT(*) FILTER (WHERE ar.status = 'A')     AS absent,
  COUNT(*) FILTER (WHERE ar.status = 'E')     AS excused,
  COUNT(*) FILTER (WHERE ar.status = 'M')     AS medical,
  SUM(s.duration_min)                         AS minutes_held,
  SUM(s.duration_min) FILTER (WHERE ar.status IN ('P','L')) AS
minutes_attended
FROM attendance_records ar
JOIN sessions s ON s.id = ar.session_id AND s.state IN ('submitted','locked')
JOIN classes  c ON c.id = s.class_id
GROUP BY ar.campus_id, ar.student_id, s.class_id, c.term_id;
```

7.3 SQL for staff, groups, curriculum tags and rotations

```
CREATE TABLE designations (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  campus_id uuid NOT NULL REFERENCES campuses(id),
  category text NOT NULL CHECK (category IN
    ('teaching','clinical','academic_support','technical','administrative')),
  name text NOT NULL,
```

```
  rank_order int NOT NULL,
  UNIQUE (campus_id, category, name)
);

CREATE TABLE staff (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  campus_id uuid NOT NULL REFERENCES campuses(id),
  user_id uuid REFERENCES users(id),
  staff_code text NOT NULL,
  category text NOT NULL,
  designation_id uuid NOT NULL REFERENCES designations(id),
  primary_department_id uuid NOT NULL REFERENCES departments(id),
  unit_department_id uuid REFERENCES departments(id),
  name text NOT NULL,
  cnic text NOT NULL CHECK (cnic ~ '^[0-9]{13}$'),
  mobile text NOT NULL,
  official_email text NOT NULL,
  employment_type text NOT NULL CHECK (employment_type IN
    ('permanent','contract','visiting','part_time','adjunct')),
  joining_date date NOT NULL,
  pmdc_no text,
  pmdc_expiry date,
  status text NOT NULL DEFAULT 'active',
  -- remaining profile columns as in the catalogue
  UNIQUE (campus_id, staff_code),
  UNIQUE (campus_id, cnic)
);

CREATE TABLE department_heads (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  campus_id uuid NOT NULL REFERENCES campuses(id),
  department_id uuid NOT NULL REFERENCES departments(id),
  staff_id uuid NOT NULL REFERENCES staff(id),
  start_date date NOT NULL,
  end_date date,
  order_no text
);
CREATE UNIQUE INDEX uq_one_current_head ON department_heads (department_id)
  WHERE end_date IS NULL;

CREATE TABLE curriculum_blocks (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  campus_id uuid NOT NULL REFERENCES campuses(id),
  version_id uuid NOT NULL REFERENCES curriculum_versions(id),
  block_no int NOT NULL,
  name text NOT NULL,
```

```
  year_of_study int NOT NULL,
  UNIQUE (version_id, block_no)
);

CREATE TABLE curriculum_modules (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  campus_id uuid NOT NULL REFERENCES campuses(id),
  block_id uuid NOT NULL REFERENCES curriculum_blocks(id),
  code text NOT NULL,
  name text NOT NULL,
  sort_order int NOT NULL,
  planned_weeks int,
  is_spiral boolean NOT NULL DEFAULT false
);

CREATE TABLE group_sets (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  campus_id uuid NOT NULL REFERENCES campuses(id),
  batch_id uuid NOT NULL REFERENCES batches(id),
  term_id uuid NOT NULL REFERENCES academic_terms(id),
  name text NOT NULL,
  purpose text NOT NULL CHECK (purpose IN ('teaching','clinical'))
);

CREATE TABLE student_groups (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  campus_id uuid NOT NULL REFERENCES campuses(id),
  group_set_id uuid NOT NULL REFERENCES group_sets(id),
  name text NOT NULL,
  code text NOT NULL,
  colour text,
  UNIQUE (group_set_id, code)
);

CREATE TABLE student_group_members (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  campus_id uuid NOT NULL REFERENCES campuses(id),
  group_set_id uuid NOT NULL REFERENCES group_sets(id),
  group_id uuid NOT NULL REFERENCES student_groups(id),
  student_id uuid NOT NULL REFERENCES students(id),
  effective_from date NOT NULL,
  effective_to date
);
-- one current group per student per group set
CREATE UNIQUE INDEX uq_one_group_per_set ON student_group_members
(group_set_id, student_id)
```

```
  WHERE effective_to IS NULL;

ALTER TABLE sections ADD COLUMN student_group_id uuid REFERENCES
student_groups(id);

CREATE TABLE teaching_slots (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  campus_id uuid NOT NULL REFERENCES campuses(id),
  batch_id uuid NOT NULL REFERENCES batches(id),
  slot_date date NOT NULL,
  start_time time NOT NULL,
  end_time time NOT NULL CHECK (end_time > start_time),
  recurrence_rule text,
  rotate_groups boolean NOT NULL DEFAULT false,
  until_date date
);

CREATE TABLE teaching_slot_assignments (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  campus_id uuid NOT NULL REFERENCES campuses(id),
  slot_id uuid NOT NULL REFERENCES teaching_slots(id),
  group_id uuid NOT NULL REFERENCES student_groups(id),
  section_id uuid NOT NULL REFERENCES sections(id),
  staff_id uuid NOT NULL REFERENCES staff(id),
  venue text,
  UNIQUE (slot_id, group_id)
);

CREATE TABLE clinical_rotations (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  campus_id uuid NOT NULL REFERENCES campuses(id),
  batch_id uuid NOT NULL REFERENCES batches(id),
  group_id uuid NOT NULL REFERENCES student_groups(id),
  department_id uuid NOT NULL REFERENCES departments(id),
  class_id uuid NOT NULL REFERENCES classes(id),
  section_id uuid NOT NULL REFERENCES sections(id),
  start_date date NOT NULL,
  end_date date NOT NULL CHECK (end_date >= start_date),
  lead_supervisor_staff_id uuid REFERENCES staff(id),
  -- a group cannot be posted to two departments at once
  EXCLUDE USING gist (group_id WITH =,
    daterange(start_date, end_date, '[]') WITH &&)
);

ALTER TABLE sessions
  ADD COLUMN clinical_subtype text,
```

```
  ADD COLUMN block_id uuid REFERENCES curriculum_blocks(id),
  ADD COLUMN module_id uuid REFERENCES curriculum_modules(id),
  ADD COLUMN tag_overridden boolean NOT NULL DEFAULT false,
  ADD COLUMN location_id uuid REFERENCES hospital_locations(id),
  ADD COLUMN slot_assignment_id uuid REFERENCES
teaching_slot_assignments(id),
  ADD COLUMN rotation_id uuid REFERENCES clinical_rotations(id),
  ADD COLUMN entered_from_paper boolean NOT NULL DEFAULT false;
```

Student clash rule (server, inside the session-create transaction): before inserting a session, the server checks that none of the roster's students already has a live session overlapping that time in any section; the same check runs for the teacher (`conducted_by`) and the venue. The database constraint on `sessions` still stops one section from overlapping itself, while groups of the same year class may be taught in parallel.

7.4 How the tables work together

- **Roster snapshot:** when a session is created, the server inserts one `attendance_records` row (default status from the bulk action) for every student whose enrolment in that section covers the session date. The register is therefore frozen to who was enrolled that day; later moves never rewrite history.
- **Percentages** come from `v_student_class_attendance` plus one server function, `calculateAttendance(counts, campusSettings)`, used by every screen and report so the maths is never duplicated. If reports slow down at scale, the view becomes a materialized view refreshed after each submit.
- **Locking:** a session is treated as locked when `state = 'locked'` **or** when `state = 'submitted'` and the lock window has passed. The server checks this on every write; a scheduled job (or the first read after the window) flips the state and sets `locked_at`.
- **Concurrency:** every session update sends the `version` it loaded; the server updates `WHERE id = $1 AND version = $2` and increments it. Zero rows updated = someone else saved first.
- **Storage estimate:** one record row is roughly 150–200 bytes with indexes. ABWA Medical College at about 600 students × 5 sessions a day × 180 teaching days ≈ 540,000 rows a year ≈ 100 MB. The 1 GB Starter Tier fits a pilot year; the whole Village will need the upgraded instance.
- **Future modules** add their own tables (timetable_slots, exams, fee_invoices) referencing the same `campus_id`, `student_id`, `class_id` and `section_id`.

8. Server API design

The browser never talks to PostgreSQL directly; it calls a versioned REST API on the Node.js server (`/api/v1`), and every call passes through the same three gates: verify the Firebase ID token, load the user's role assignments, check the action against the permission matrix for the requested `campus_id` .

8.1 Request pipeline

1. **Authenticate:** `Authorization: Bearer <Firebase ID token>` → verified with firebase-admin → `users` row found by `firebase_uid` (first sign-in of an invited email links the uid).
2. **Scope:** header `X-Campus-Id` (the campus selected in the switcher) must be a campus the user holds a role in; otherwise 403.
3. **Authorise:** `can(user, action, resource)` from one permissions module. Teacher checks include "is this faculty member on `class_teachers` , `section_teachers` or the session's substitute".
4. **Validate:** request body checked against a schema (e.g. Zod); clear field errors with HTTP 422.
5. **Execute** inside a database transaction where more than one table changes; write `audit_logs` in the same transaction.
6. **Respond** with a consistent shape: `{ data, meta }` on success, `{ error: { code, message, fields } }` on failure. Codes: 400, 401, 403, 404, 409 (conflict / overlap / version), 422, 500.

8.2 Endpoint catalogue (Phase 1)

Area	Method and path	Purpose	Who
Session	GET /me	Profile, roles, campuses	All
Campuses	GET, POST /campuses · PATCH /campuses/:id	List, create, edit	Super Admin
Settings	GET, PATCH /campuses/:id/settings	Attendance rules	Super / Campus Admin
Users	GET, POST /users · POST /users/:id/roles · DELETE /users/:id/roles/:roleId	Invite users, grant and revoke roles	Super / Campus Admin

Area	Method and path	Purpose	Who
Academics	CRUD /departments, /programs, /batches, /terms, /calendar-events, /courses	Academic structure	Campus Admin
Academics	POST /batches/:id/promote	Year rollover	Campus Admin
People	CRUD /faculty, /students · POST /students/:id/status	Profiles and status changes	Campus Admin
Imports	POST /imports/students · POST /imports/faculty · GET /imports/:id	Upload, validate, commit, error report	Campus Admin
Classes	CRUD /classes · POST /classes/copy-from-term	Classes and rollover copy	Campus Admin
Classes	PUT /classes/:id/teachers	Assign teachers and lead	Campus Admin
Sections	CRUD /classes/:id/sections · POST /classes/:id/sections/auto-split	Create and auto-split	Campus Admin
Enrolment	POST /classes/:id/enrol · POST /enrollments/move · POST /enrollments/:id/end	Enrol, move between sections, end	Campus Admin
Teacher	GET /my/sections?date=	Sections the teacher can mark today and overall	Teacher
Sessions	POST /sessions	Create session + roster snapshot (draft)	Teacher, Admin
Sessions	GET /sessions/:id	Session with records	Teacher (own), Admin
Sessions	PUT /sessions/:id/records	Save marks (body: version, records[])	Teacher (own, before lock), Admin
Sessions	POST /sessions/:id/submit · /cancel · /admin-edit	State changes	As per matrix
Sessions	POST /sessions/combined	One merged roster across several sections	Teacher

Area	Method and path	Purpose	Who
Corrections	POST /sessions/:id/corrections · GET /corrections · POST /corrections/:id/approve · /reject	Correction workflow	Teacher, Admin, HOD
Reports	GET /reports/daily-summary, /section-register, /student-card, /shortage, /class-log, /teacher-activity, /corrections, /village-overview	Report data (JSON)	As per matrix
Exports	GET /reports/:name/export?format=pdf|xlsx	File download	As per matrix
Audit	GET /audit-logs	Filterable history	Super / Campus Admin

8.3 The two critical endpoints

POST /sessions (create and snapshot) runs as one transaction:

1. Check the teacher may mark this section and the date is within the back-date limit.
2. Warn (not block) if the date is a holiday or closure; require `reason` if the client confirms.
3. Insert the session (overlap is rejected by the database constraint → 409 with the clashing session's time).
4. Insert one `attendance_records` row per currently enrolled student with the chosen default status (P or A).
5. Return the session with the roster sorted by roll number.

POST /sessions/:id/submit runs as one transaction:

1. Reject if the caller lacks rights or the session is locked/cancelled.
2. Check `version` matches; else 409 "updated by someone else".
3. Validate E/M rows have remarks.
4. Set `state = 'submitted'`, `submitted_at = now()`, increment `version`.
5. Write an audit entry with the counts.
6. Return the counts and the lock deadline. The same `idempotency_key` sent twice returns the first result.

9. Security, audit, backups and non-functional requirements

Security is enforced on the server and in the database, never only by hiding buttons; a published AI Studio app is reachable by anyone on the internet, so every API call must prove who the user is and what campus they may touch.

9.1 Security

- **Two layers of campus isolation.** Layer 1: every query in the server's data layer takes `campusId` as a required argument and adds `WHERE campus_id = $campusId`; a lint rule or code review rejects queries without it. Layer 2 (recommended once stable): PostgreSQL Row-Level Security policies on campus-owned tables, with the server setting `SET LOCAL app.campus_ids = '…'` per request, so a missed filter still returns nothing.
- **Database credentials** live only in AI Studio's Secrets panel and the server environment, never in browser code.
- **Invite-only sign-in.** A Firebase account that is not in `users` (and not invited) sees "Access not granted, contact your administrator". Disabled users are rejected even with a valid token.
- **Sensitive fields** (CNIC, phone, address, DOB) are removed from API responses unless the caller is Campus Admin or Super Admin.
- **Rate limiting** on sign-in and export endpoints; request size limits on imports (e.g. 5 MB, 5,000 rows).
- **Parameterised queries only** (the ORM does this); no SQL built from strings.
- **Uploads** (photos, Excel files) validated by type and size; photos stored in Cloud Storage, not in the database.
- **HTTPS only**; secure, HTTP-only cookies if sessions are used; CORS limited to the app's own domain.

9.2 Audit log

Written in the same transaction as the change, append-only (the app's database user has no UPDATE or DELETE on `audit_logs`).

Event	Recorded
Sign-in, failed sign-in	user, time, IP
Role granted / revoked	who, to whom, role, scope
Campus settings changed	before, after

Event	Recorded
Student status changed	from, to, effective date, reason
Section move, enrolment start / end	student, from, to, date
Session created, submitted, cancelled	counts, teacher, time
Any change after lock	each student's from/to, reason, approver
Correction approved / rejected	request, decision, comment
Bulk import	file name, rows ok / failed
Export of student data	report, filters, who

9.3 Backups and data safety

The AI Studio developer-edition database has **no built-in backup or recovery**, so ABWA must add its own before relying on it.

1. Build and pilot on the developer instance, but link a **billing-enabled ABWA Google Cloud project** in the "Enable database" settings from the start, so the instance can be resized and managed.
2. Add a **nightly export**: a scheduled job that runs `pg_dump` (or Cloud SQL export) to a Cloud Storage bucket with 30-day retention. Ask AI Studio for this in Prompt 11.
3. Add an in-app **"Download full backup"** button for the Super Admin (Excel/CSV of all tables) as a second safety net.
4. Before the whole Village goes live, plan migration to a standard Cloud SQL instance with automated backups and point-in-time recovery.
5. Test a restore once per term into a scratch database.

9.4 Non-functional requirements

Area	Requirement
Speed	Roster of 150 loads in under 2 s on 4G; mark-and-submit in under 60 s; reports for one batch in under 5 s
Mobile	Teacher screens fully usable at 360 px width; tap targets at least 44 px
Unreliable network	Draft marks kept on the device and re-sent automatically; submit is idempotent

Area	Requirement
Cold start	Starter-tier database pauses when idle; first request of the morning may take a few seconds, so show a friendly loader
Scale	Design for 5+ campuses, 10,000 students, 1,000 faculty, ~2,000 sessions a day; pilot on AMC alone
Time	Dates and times shown in Asia/Karachi, DD-MMM-YYYY, 12-hour clock; audit timestamps in UTC
Exports	A4 PDF registers with campus logo and header; Excel with raw codes and readable labels
Accessibility	Status always shown as a letter as well as a colour; keyboard-usable admin tables
Branding	ABWA Knowledge Village identity in the shell; each campus's logo and name in its header
Language	English UI; all text in one strings file so Urdu can be added
Maintainability	Code split by module; one permissions module; one attendance-calculation function with unit tests; migrations in version control (GitHub export)

10. Screen inventory

Phase 1 needs 36 screens across three roles; the navigation shown to a user depends on their role in the currently selected campus.

App shell: left sidebar (collapses to bottom nav on phone), top bar with campus switcher, user menu, and ABWA Knowledge Village branding. Sidebar items are generated from a module registry so new modules add menu entries without touching existing code.

#	Screen	Role	Main contents
1	Sign in	All	Email/password, Google sign-in, forgot password
2	Village dashboard	Super Admin	Campus cards with today's attendance %, active students, unmarked sections

#	Screen	Role	Main contents
3	Campuses	Super Admin	List, create/edit campus, settings, appoint admins
4	Users & roles	Super Admin, Campus Admin	User list, invite, assign/revoke roles, disable
5	Campus dashboard	Campus Admin, Principal	KPIs: sessions today, present %, shortage count; unmarked sections; pending corrections; trend chart (30 days)
6	Setup checklist	Campus Admin	Gaps: classes without teachers, sections without students, students without sections
7	Departments	Campus Admin	CRUD, HOD
8	Programs & batches	Campus Admin	CRUD, promote batch
9	Academic terms & calendar	Campus Admin	Terms, holidays, exam blocks (calendar view)
10	Courses	Campus Admin	CRUD, filter by year
11	Faculty	Campus Admin	List, profile, import, create login
12	Students	Campus Admin	List with filters (batch, status), profile, import
13	Student profile	Campus Admin, Teacher (read)	Details, enrolled classes and sections, attendance card
14	Classes	Campus Admin	List by batch/term, create, copy from previous term
15	Class detail	Campus Admin	Teachers, sections, enrolment, sessions history
16	Section manager	Campus Admin	Auto-split helpers, drag-and-drop between sections, teacher per section
17	My Classes (today)	Teacher	Today's sections, last session date, Take attendance button
18	Take attendance	Teacher	Session header (date, time, topic), roster grid, mark-all, counters, save/submit

#	Screen	Role	Main contents
19	Session detail	Teacher, Admin	Read view, edit (if allowed), request correction, audit trail
20	Section register	Teacher, Admin	Month grid students × sessions, percentages, export
21	Correction requests	Campus Admin, HOD	Queue, approve/reject
22	Reports	Campus Admin, Principal, HOD	All reports from 5.6 with filters and export
23	Campus settings	Campus Admin	Threshold, lock window, late/excused rules, ID patterns
24	Audit log	Super Admin, Campus Admin	Searchable change history

Additional screens for registration, groups, curriculum tags and clinical teaching

#	Screen	Role	Main contents
25	Student registration form	Admissions, Campus Admin	Multi-step form (Admission, Identity, Contact, Guardians, Previous education, Documents), autosave, completeness bar
26	Verification queue	Campus Admin	Submitted records, document checklist with previews, Verify / Return with note, Activate (issues numbers, assigns groups)
27	Staff registration form and directory	Campus Admin, HR	Multi-step staff form; directory filtered by department, category, designation; PMDC expiry flags
28	Designations	Campus Admin	Categories and designations with rank order
29	Department page	Campus Admin; HOD read-only	Details, Select head with history, staff list, classes, hospital locations

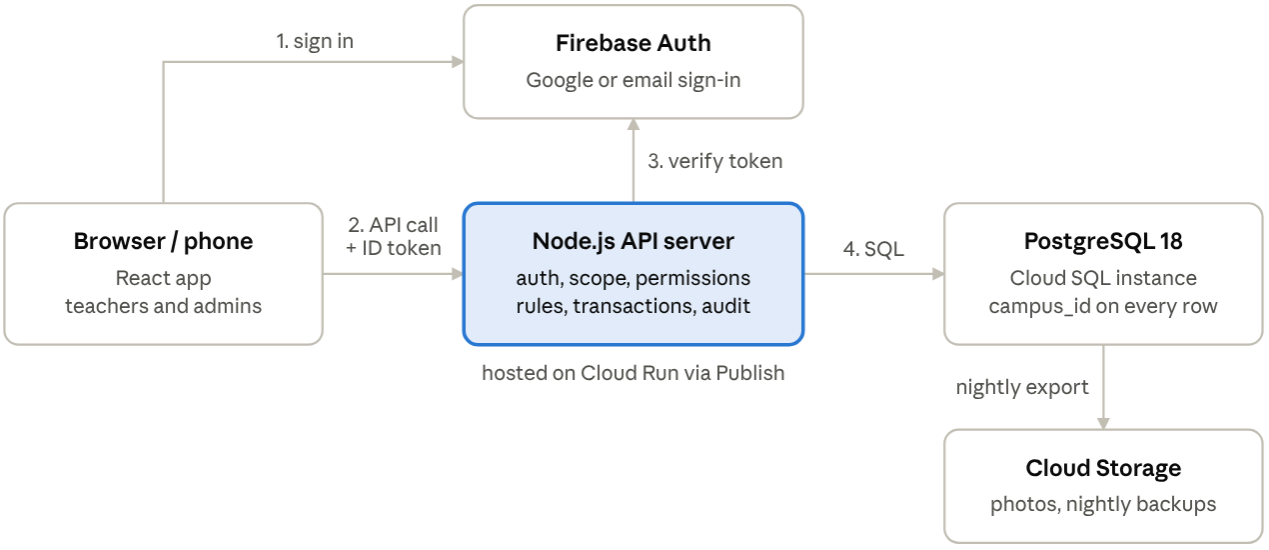
#	Screen	Role	Main contents
30	Curriculum	Campus Admin	Versions, blocks, modules; per-batch block/module calendar (timeline view)
31	Groups	Campus Admin	Group sets per year class, auto-fill wizard, drag-and-drop between groups, unassigned list
32	Teaching slot planner	Campus Admin	Week grid: time rows × group columns; pick class, teacher, venue per cell; clash warnings; repeat and rotate
33	Rotation planner and board	Campus Admin; HOD and Principal read-only	Groups × weeks grid coloured by department; supervisors; this week's board
34	My Rotations	Clinical supervisor	Today's rotation groups, Take attendance with sub-type and ward
35	Module / block attendance report	Campus Admin, HOD	Student × module or block percentages per group
36	Rotation completion report	Campus Admin, HOD	Each student's posting attendance vs clinical threshold, flags for repeat

Visual direction: clean, institutional, high-contrast; status chips P green, A red, L amber, E/M blue/grey with the letter always shown; tables with sticky headers; empty states that tell the admin what to set up next.

11. Architecture on Google AI Studio and what to prepare

One AI Studio app holds both the React front end and the Node.js server; AI Studio creates the PostgreSQL database and Firebase sign-in when you ask for them, and Publish puts the whole app on Cloud Run.

The browser never touches the database; the server checks every request



ABWA CMS architecture on Google AI Studio

Only the server holds the database password (in AI Studio's Secrets panel); the browser holds just a short-lived sign-in token.

11.1 What you prepare before Prompt 1

- [ ] **Google account:** decide whether to build under a personal Google account (eligible for the free Starter Tier) or an ABWA Google Cloud project with billing. Workspace accounts are not eligible for the free tier.
- [ ] **Region:** note the closest available Google Cloud region to Pakistan; you choose it once in the "Enable database" card and cannot change it.
- [ ] **Settings decided:** threshold, warning band, late rule, excused rule, calculation basis, lock window, back-date limit, ID formats (section 14).
- [ ] **Logos:** ABWA Knowledge Village and ABWA Medical College logos as PNG/SVG.
- [ ] **Data files** in Excel, one sheet each, using the column lists below.
- [ ] **Pilot group chosen:** one batch (e.g. 1st Year MBBS) with 3–5 teachers who will use it first.

11.2 Data file templates

File	Columns (in this order)
Departments	code, name, type (basic_science / para_clinical / clinical / support), location (college / hospital), parent_code

File	Columns (in this order)
Hospital locations	department_code, name, kind (ward / opd / ot / er / icu / lab / skills_lab / room), floor, capacity
Curriculum modules	block_no, block_name, year_of_study, module_code, module_name, order, planned_weeks, is_spiral
Courses	code, title, department_code, year (1–5), components (lecture;practical;sgd;clinical…)
Staff	staff_code, name, father_or_husband_name, gender, dob, cnic, category, designation, primary_department_code, secondary_department_codes, employment_type, joining_date (DD-MM-YYYY), mobile, official_email, pmdc_no, pmdc_expiry
Students	admission_category, merit_no, mdcat_roll_no, mdcat_score, name, father_name, gender, dob, cnic, nationality, domicile_province, domicile_district, mobile, email, permanent_address, city, guardian_name, guardian_relation, guardian_mobile, matric_marks, matric_total, fsc_marks, fsc_total, batch, admission_date
Groups	reg_no, group_set_name, group_code
Classes	course_code, class_type, title, teacher_staff_codes (lead first), default_duration_min
Rotation plan	group_code, department_code, start_date, end_date, lead_supervisor_staff_code

11.3 How to work inside AI Studio

- **One prompt, one feature.** Paste one prompt from section 12, let it finish, test it in the Preview, then move on.
- **Use the Code tab to check the schema file** after any database change; ask "Show me the schema in the database" to see the live tables.
- **Use annotation mode** for visual fixes: highlight the part of the screen and describe the change.

- **Save a version after every working step** (export to GitHub or download the code) so you can roll back if a later prompt breaks something.
- **Don't let the agent recreate tables.** If it proposes dropping and recreating a table that already has data, stop and ask for a migration instead.
- **Keep a test account per role** (Super Admin, Campus Admin, Teacher) so you can sign in as each one.

12. Prompt-by-prompt build guide (Google AI Studio + PostgreSQL)

Fifteen prompts take the system from an empty AI Studio app to a published attendance system for ABWA Medical College; prompts 1–6A build registration and the setup attendance needs (staff and students are registered before any class exists), prompt 7 is the first day teachers can mark attendance, and 8A brings in hospital rotations.

#	Prompt	Result you test	Rough effort
1	Foundation and database	App shell, sign-in, all 46 Phase 1 tables, seeded Village + 2 campuses	1 session
2	Users, roles, campus settings	Invite users, grant roles, campus switcher, settings screen	1 session
3	Departments, heads, academics, curriculum	Department types and hospital locations, MBBS, batches, terms, blocks and modules, courses	1–2 sessions
4A	Teachers and staff registration	Full staff register, designations, select department heads	1–2 sessions
4B	Student registration	Draft → verify → activate, documents, imports	2 sessions
5	Classes and teachers	Classes created, teachers assigned, default section, enrolment	1 session
6	Sections	Auto-split, moves, per-section teachers	1 session
6A	Groups and parallel teaching slots	Group sets, linked sections, slot planner, clash checks	1–2 sessions
7	Taking attendance	Teacher marks and submits from a phone	1–2 sessions

#	Prompt	Result you test	Rough effort
8	Lock, cancel, corrections	Lock window, correction approvals, combined sessions, audit	1–2 sessions
8A	Clinical rotations and hospital attendance	Rotation plan, My Rotations, ward attendance, completion report	1–2 sessions
9	Calculation and core reports	Section register, student card, shortage list, module / block levels, unit tests	1–2 sessions
10	Dashboards and exports	Campus and Village dashboards, PDF/Excel	1 session
11	Hardening and backups	Security review, row-level security, nightly export	1–2 sessions
12	Pilot and publish	Real data loaded, published, help page	1 session

How to use each step: paste the **Context block** below, then the step's prompt, in the same message. Wait for the build to finish, run the step's checks, save a version, and only then continue. If a check fails, use the troubleshooting prompts in section 13 before moving on.

Context block (paste at the top of every prompt from 2 to 12)

```
CONTEXT (do not change the existing architecture): This is ABWA CMS, a multi-campus Campus Management System for ABWA Knowledge Village, Pakistan. Stack: React + TypeScript + Tailwind front end, Node.js server API (/api/v1), Cloud SQL for PostgreSQL via the existing schema and migrations, Firebase Authentication. Hierarchy: Organisation > Campus > Department > Program > Batch > Term > Course > Class > Section > Session > Attendance record. Every campus-owned table has campus_id and every query filters by it. Every class has at least one section; attendance is kept per section. People are registered first: students through the Student Registration module (only Active students appear on rosters) and all employees in one staff table (only teaching and clinical staff can teach). Departments have a type, a location (college or hospital) and a head; department heads can view and approve but never assign classes, teachers, sections, groups, rotations or substitutes. MBBS year classes are divided into group sets; sections can be linked to groups so rosters follow group membership; groups can be taught different subjects at the same time through teaching slots; clinical groups rotate through hospital departments. Every session is tagged with the UHS curriculum Block and Module for its date. A student, a teacher and a venue can never be in two overlapping sessions. All permission checks go through the single permissions module; all attendance maths through the single calculateAttendance function. Never drop or recreate tables that hold data: change the schema only with a new migration. Never hard-delete; archive. Write an audit_logs row for every sensitive change. Dates shown in Asia/Karachi as DD-MMM-YYYY, 12-hour time. Keep UI text in the strings file. After finishing, list the files changed, any migration added, and what I should test.
```

Prompt 1 — Foundation and database

Goal: a running app with sign-in, the full Phase 1 database, and the code structure every later prompt builds on. When the "Enable database" card appears, open its settings, choose your project and region deliberately, then enable Cloud SQL.

```
Build the foundation of "ABWA CMS", a multi-campus Campus Management System for ABWA Knowledge Village (Pakistan). Use a Cloud SQL PostgreSQL database and Firebase Authentication (Google sign-in AND email/password). This step builds the foundation only; later prompts add features.

1. ARCHITECTURE
- React + TypeScript + Tailwind front end; Node.js server exposing a REST API under /api/v1. The browser never connects to the database.
- Folders: server/core (auth middleware, campus scope, permissions, audit, errors), server/modules/<module> (routes, services, queries), server/db (schema, migrations, seed), client/core (layout, auth context, campus
```

```
switcher, api client, shared UI), client/modules/<module>, shared/types.
- Request pipeline on every API call: verify Firebase ID token with firebase-
admin -> load users row by firebase_uid -> read X-Campus-Id header and
confirm the user holds a role there -> permissions check -> validate body
with zod -> DB transaction when more than one table changes -> audit_logs row
in the same transaction -> respond {data, meta} or {error:
{code,message,fields}}.
- MODULE REGISTRY: each client module exports {id, name, icon, routes,
sidebarItems, requiredRoles}; the sidebar is built from it.
- Versioned migrations for every schema change. UUID primary keys
(gen_random_uuid()), snake_case, created_at/created_by/updated_at/updated_by
and status on every table, campus_id on every campus-owned table, indexed
foreign keys, no hard deletes. Enable the btree_gist extension.

2. DATABASE - create ALL these tables now, even if screens come later:
TENANCY: organizations; campuses (org_id, name, code unique per org, type
college|university, term_type annual|semester, timezone default Asia/Karachi,
logo_url); campus_settings (campus_id PK, threshold_pct 75, warning_pct 80,
clinical_threshold_pct 75, lock_hours 24, backdate_days 7, late_rule
present|n_lates_one_absence, late_to_absent_n 3, excused_rule
exclude|present, calc_basis sessions|hours, hod_can_approve false,
count_emergency_duty true, allow_adhoc_group_sessions true, reg_no_pattern,
roll_no_pattern, staff_code_pattern).
IDENTITY: users (firebase_uid unique, email unique, name, phone, photo_url,
is_active, last_login_at); role_assignments (user_id, role
SUPER_ADMIN|CAMPUS_ADMIN|PRINCIPAL|HOD|TEACHER|STUDENT|PARENT, campus_id
nullable, department_id nullable, unique together).
DEPARTMENTS: departments (code unique per campus, name, dept_type
basic_science|para_clinical|clinical|support, location college|hospital,
parent_department_id for units, hod_staff_id); department_heads
(department_id, staff_id, start_date, end_date, order_no; partial unique
index: one open row per department); hospital_locations (department_id, name,
kind ward|opd|ot|er|icu|lab|skills_lab|room, floor, capacity).
ACADEMICS: programs (department_id, name, code, duration_years); batches
(program_id, intake_year, session_label, current_term_id,
curriculum_version_id, unique program+year); academic_terms (start_date,
end_date, check end>start); batch_terms (batch_id, term_id,
year_or_semester_no); calendar_events (start_date, end_date, type
holiday|exam|closure|event, title); courses (department_id, program_id, code
unique per campus, title, year_or_semester, credit_hours).
CURRICULUM: curriculum_versions (name, affiliating_body, effective_from);
curriculum_blocks (version_id, block_no unique per version, name,
year_of_study); curriculum_modules (block_id, code, name, sort_order,
planned_weeks, is_spiral); batch_curriculum_dates (batch_id, block_id,
module_id nullable, start_date, end_date, exam_date).
STAFF: designations (category
```

teaching|clinical|academic_support|technical|administrative, name, rank_order, unique campus+category+name); staff (user_id nullable, staff_code unique per campus, category, designation_id, primary_department_id, unit_department_id, name, father_or_husband_name, gender, dob, cnic 13 digits unique per campus, mobile, official_email, personal_email, permanent_address, current_address, emergency_name, emergency_relation, emergency_mobile, employment_type permanent|contract|visiting|part_time|adjunct, bps_grade, joining_date, contract_end_date, reports_to_staff_id, pmdc_no, pmdc_expiry, experience_years, photo_url, status); staff_departments (staff_id, department_id); staff_qualifications (staff_id, degree, specialty, institution, year); staff_documents (staff_id, doc_type, file_url, verified_by, verified_at).
STUDENTS: students (program_id, batch_id, user_id nullable, registration_state draft|submitted|verified|returned|active, reg_no and roll_no nullable until active then unique per campus / per batch, name, father_name, gender, dob, cnic, nationality, passport_no, domicile_province, domicile_district, blood_group, mobile, whatsapp, email, permanent_address, current_address, city, admission_date, admission_category, merit_no, mdcat_roll_no, mdcat_score, aggregate_pct, uhs_reg_no, uhs_reg_date, hostel_resident, photo_url, verified_by, verified_at, activated_at, status); student_guardians (student_id, name, relation, cnic, occupation, mobile, email, is_emergency_contact); student_academic_records (student_id, level matric|o_level|fsc|a_level, board, year, roll_no, obtained, total, ibcc_no); student_documents (student_id, doc_type, file_url, original_seen, verified_by, verified_at); student_status_history.
GROUPS: group_sets (batch_id, term_id, name, purpose teaching|clinical); student_groups (group_set_id, name, code unique per set, colour); student_group_members (group_set_id, group_id, student_id, effective_from, effective_to; partial unique index: one open membership per student per set).
TEACHING: classes (course_id, batch_id, term_id, department_id, class_type lecture|tutorial|practical|sgd|clinical|skills_lab|demonstration, title, default_duration_min, venue); class_teachers (class_id, staff_id, is_lead); sections (class_id, name unique per class, student_group_id nullable, capacity, is_default, venue); section_teachers (section_id, staff_id); enrollments (class_id, section_id, student_id, effective_from, effective_to, source manual|group_link|rotation; partial unique index: one open enrolment per class+student); teaching_slots (batch_id, slot_date, start_time, end_time, recurrence_rule, rotate_groups, until_date); teaching_slot_assignments (slot_id, group_id, section_id, staff_id, venue, unique slot+group).
CLINICAL: clinical_rotations (batch_id, group_id, department_id, class_id, section_id, start_date, end_date, lead_supervisor_staff_id, EXCLUDE constraint so a group cannot be in two rotations with overlapping dates); rotation_supervisors (rotation_id, staff_id).
ATTENDANCE: sessions (class_id, section_id, session_date, start_time, end_time, generated duration_min, session_type, clinical_subtype, topic NOT

```
NULL, conducted_by staff_id, is_substitute, venue, block_id, module_id,
tag_overridden, location_id, slot_assignment_id, rotation_id,
entered_from_paper, state draft|submitted|locked|cancelled, cancel_reason
required when cancelled, combined_group_id, submitted_at, locked_at, version,
idempotency_key unique, EXCLUDE constraint so live sessions of one section
cannot overlap); attendance_records (session_id, student_id, status
P|A|L|E|M, remark required for E and M, marked_by, marked_at, unique
session+student); correction_requests; correction_request_items.
AUDIT: audit_logs (append-only); import_jobs.
Also create the view v_student_class_attendance (held, present, late, absent,
excused, medical, minutes_held, minutes_attended per student per class from
submitted/locked sessions).

3. AUTH AND ROLES
- Sign-in with Google and email/password. Only invited users get in; unknown
emails see "Access not granted, contact your administrator".
- First-run setup: if no SUPER_ADMIN exists, the first signed-in user may
become Super Admin and create "ABWA Knowledge Village".
- Permissions module can(user, action, resource): SUPER_ADMIN everything;
CAMPUS_ADMIN everything in own campus; PRINCIPAL read-only campus; HOD read
own department and optionally approve corrections, but NEVER create/assign
classes, teachers, sections, groups, rotations or substitutes; TEACHER own
classes, sections and rotations only; STUDENT/PARENT defined, no screens yet.

4. APP SHELL
- Sidebar (bottom navigation on phones), top bar with ABWA Knowledge Village
branding, campus switcher, user menu.
- Placeholder pages: Dashboard, Setup, Registration (Students, Staff),
Departments, Academics, Curriculum, Groups, Classes, Rotations, Attendance,
Reports, Settings, Audit log.
- Clean institutional design; status chips show the letter: P green, A red, L
amber, E blue, M grey. All UI text in one strings file. Dates in Asia/Karachi
as DD-MMM-YYYY.

5. SEED DATA (seed script, safe to run once)
Organisation ABWA Knowledge Village; campuses ABWA Medical College (AMC,
college, annual) and ABWA Nursing College (ANC, college, annual) with default
campus_settings; for AMC the designation list from the spec and curriculum
version "UHS Modular Integrated Curriculum 2K23" with Year 1 blocks 1-3. No
people or classes yet.

When done, show me the schema in the database, list every file created, and
tell me how to make myself Super Admin and invite a Campus Admin.
```

Check before moving on

- [ ] Ask "Show me the schema in the database" and confirm all 46 tables and the view exist.
- [ ] You can sign in with Google and with email/password; an un-invited account is refused.
- [ ] You are Super Admin and see both campuses in the switcher.
- [ ] The Code tab shows a migrations folder and a permissions module.
- [ ] Save a version (GitHub export or download).

Prompt 2 — Users, roles and campus settings

Goal: the Super Admin can appoint the Medical College administrator, and campus rules are editable.

```
[Paste the Context block here]

Build the Users & Roles and Campus Settings features.

1. USERS & ROLES (Super Admin: all campuses; Campus Admin: own campus only, cannot grant SUPER_ADMIN)
- Users list with search and filters (role, campus, active/disabled).
- "Invite user": email, name, phone, role, campus, optional department (for HOD). Creates a users row with is_active true and no firebase_uid; when that email first signs in, link the firebase_uid automatically.
- Grant / revoke roles on a user detail page; a user can hold several roles in several campuses.
- Disable / re-enable a user (disabled users are refused by the API even with a valid token). Never delete users.
- Audit every invite, role change, disable and enable.

2. CAMPUSES (Super Admin)
- Campuses list and form: name, code, type, term type, address, logo upload, status active/archived.

3. CAMPUS SETTINGS (Campus Admin, Super Admin)
- Form for campus_settings with plain-language help under each field: attendance threshold %, warning band %, late rule (late counts as present / N lates = 1 absence, with N), excused and medical leave rule (exclude from total / count as present), calculation basis (sessions / contact hours), lock window in hours, back-date limit in days, HOD can approve corrections, registration and roll number patterns (e.g. AMC-MBBS-{YY}-{###}).
- Show a live preview: "With these settings, a student with 30 P, 3 L, 3 A, 2 M out of 38 sessions = X% (OK / At risk / Shortage)".
- Audit every settings change with before/after values.
```

```
4. CAMPUS SWITCHER
- Remember the last selected campus per user; send it as X-Campus-Id on every
request; reload data when it changes.
```

Check: invite a test Campus Admin for AMC and a test user for ANC; sign in as the AMC admin and confirm ANC is invisible everywhere, including by typing an ANC URL. Change the threshold and see the preview update.

Prompt 3 — Departments, academics and curriculum

Goal: the Medical College's college and hospital departments, MBBS program, batches, terms, UHS blocks and modules, calendar and courses exist.

```
[Paste the Context block here]

Build the Departments, Academics and Curriculum modules for Campus Admins
(read-only for Principal; HOD read-only for own department).

1. DEPARTMENTS: list grouped by type; create / edit / archive with code
(unique per campus), name, type (basic science, para-clinical, clinical,
support), location (college or hospital), parent department for units (e.g.
Surgery Unit I under Surgery). Seed the editable list from the spec: Anatomy,
Physiology, Biochemistry (basic science, college); Pathology, Pharmacology,
Forensic Medicine, Community Medicine, Behavioural Sciences (para-clinical,
college); Medicine, Cardiology, Pulmonology, Gastroenterology, Nephrology,
Neurology, Psychiatry, Dermatology, General Surgery, Orthopaedics,
Neurosurgery, Urology, Plastic Surgery, Paediatric Surgery, Anaesthesia,
Gynaecology & Obstetrics, Paediatrics, ENT, Ophthalmology, Radiology,
Emergency Medicine (clinical, hospital); Medical Education, Examination,
Library, IT, Administration, Accounts (support, college).
2. HOSPITAL LOCATIONS for hospital departments: name, kind (ward, OPD, OT,
ER, ICU, lab, skills lab, room), floor, capacity.
3. Leave a "Head of department" area on the department page showing "Vacant";
head selection is built in the staff prompt.
4. PROGRAMS, BATCHES, TERMS: MBBS (5 years); batches per intake with session
label; academic terms with dates; link batches to terms with year number;
current term per batch.
5. CURRICULUM: curriculum versions (e.g. UHS Modular Integrated Curriculum
2K23); blocks (number across the programme, name, year of study); modules
(code, name, block, order, planned weeks, spiral flag). For each batch, a
block/module calendar with start, end and exam dates, shown as a horizontal
timeline. Link each batch to its curriculum version.
6. A shared function getCurriculumTag(batchId, date) returning the block and
module running on that date (spiral modules excluded unless chosen); it will
be used to auto-tag sessions.
7. ACADEMIC CALENDAR: month view with holidays, exam blocks and closures.
8. COURSES (disciplines): code, title, department, program, year (1-5),
component types (lecture, practical, SGD, clinical, skills lab). Filter by
year.
9. Excel import for departments, hospital locations, modules and courses with
templates, preview, row errors and import_jobs record.
10. SETUP CHECKLIST page: departments without a head, hospital departments
without locations, batches without current term, batches without a
block/module calendar, years without courses.
```

Check: confirm the seeded departments show the right type and location; add two wards to Surgery; create MBBS, batch 2026, term "2026–27" (1st Year); enter Block-1 with its modules and dates; call up a date in Foundation-I and confirm getCurriculumTag returns

Block 1 / Foundation-I. The Setup checklist lists every department head as Vacant.

Prompt 4A — Teachers and staff registration, department heads

Goal: every employee is registered with designation and department, and each department has its head.

```
[Paste the Context block here]

Build the Teachers and Staff Registration module and department head
selection (Campus Admin; HOD read-only for own department staff).

1. DESIGNATIONS page: categories (teaching, clinical, academic support,
technical, administrative) with designations and rank order; seed from the
spec (Professor, Associate Professor, Assistant Professor, Senior Lecturer,
Lecturer, Senior Demonstrator, Demonstrator, Visiting Faculty; clinical:
Senior Registrar, Registrar, Consultant, Medical Officer, House Officer; and
the support, technical and admin lists).
2. STAFF FORM (multi-step, autosave): Identity (name, father/husband name,
gender, DOB, CNIC 13 digits unique per campus, nationality, domicile, photo);
Contact (mobile, official and personal email, addresses, emergency contact);
Employment (staff code from campus pattern, category, designation filtered by
category, primary department, secondary departments, unit for hospital
departments, employment type, BPS/grade, joining date, contract end, reports
to, status with effective date); Professional for teaching and clinical staff
(PMDC number and expiry, qualifications list, experience years, subjects able
to teach); Documents (upload, verified by/date); System access (Create login:
invites official email; TEACHER role granted automatically for teaching and
clinical categories).
3. STAFF DIRECTORY: filters by department, category, designation, status;
sort by designation rank; PMDC expiry badge (red when expired, amber within
60 days); export.
4. Excel import for staff with template, mapping, validation (CNIC,
duplicates, unknown department or designation) and error file.
5. SELECT HEAD on the department page: pick from staff whose primary or
secondary department is this department; start date and order number; in one
transaction close the previous head (end date), insert department_heads row,
set departments.hod_staff_id, grant HOD role scoped to the department to the
new head and revoke it from the previous head; audit. Show head history.
6. Enforce in the permissions module: HOD can view department staff, classes,
groups, rotations and reports and (if enabled) approve corrections, but every
create/assign endpoint for classes, teachers, sections, groups, rotations and
substitutes returns 403 for HOD.
7. Only active staff in teaching or clinical categories appear in any
teacher, supervisor or substitute picker. When a staff member leaves, disable
login and list their open assignments for reassignment.
8. Reports: department strength (designations per department), PMDC expiry,
staff directory PDF/Excel.
```

Check: import your staff list; select heads for Anatomy and Surgery; sign in as the Anatomy head and confirm you can see department reports but every "Create class" or "Assign teacher" action is missing and refused by the API.

Prompt 4B — Student registration

Goal: every student of the pilot batch is registered, verified and active, ready for groups and attendance.

```
[Paste the Context block here]

Build the Student Registration module (Campus Admin; admissions staff can be given CAMPUS_ADMIN-limited "registration" permission later).

1. REGISTRATION FORM (multi-step with autosave and a completeness bar): Admission (program, batch, admission date, admission category from a configurable list, merit no, MDCAT roll no and score, aggregate %, admission letter no); Identity (name as on Matric certificate, father's name, gender, DOB, CNIC/B-Form, nationality, passport for foreign students, domicile province and district, blood group, photo); Contact (mobile, WhatsApp, email, permanent and current address, city); Guardians (one or two guardians with relation, CNIC, occupation, mobile, email; emergency contact); Previous education (Matric/O-Level and FSc/A-Level with board, year, roll no, obtained and total marks; IBCC equivalence); University registration (UHS registration number and date, optional); Residence (hostel yes/no); Documents checklist (Matric, FSc/A-Level, IBCC, MDCAT result, CNIC/B-Form, domicile, guardian CNIC, photos, character certificate, medical fitness, undertaking, admission letter) each with upload and "original seen".
2. STATES: draft -> submitted -> verified or returned (with note) -> active. Submit requires all required fields. Verification queue for Campus Admin with document previews.
3. ACTIVATE: issue registration number from the campus pattern and roll number (next in batch, editable once), set year class, assign teaching group (and clinical group when sets exist), set status Active, record activated_at; then enrol the student into all "All" sections of their year class and into sections linked to their groups. Numbers are never reused.
4. VALIDATION: CNIC/B-Form 13 digits; Pakistani or international mobile; marks not above totals; age at least 16; duplicate warning on CNIC, MDCAT roll no, or name + father name + DOB.
5. Excel import creating records as Draft or Active (admin choice) with row errors file.
6. STUDENT PROFILE: all sections, documents, status history, groups, enrolled classes, attendance card; printable A4 registration form with photo.
7. Reports: registration status summary, document-pending list, batch list export.
8. Privacy: CNIC, contact, guardians and documents visible only to CAMPUS_ADMIN and SUPER_ADMIN.
```

Check: register two students by hand through every state, import the rest of the batch, return one record and resubmit it, and confirm only Active students are counted in the batch strength.

Prompt 5 — Classes, teachers and enrolment

Goal: every class the pilot batch attends exists, with teachers and students attached.

```
[Paste the Context block here]

Build the Classes module for Campus Admins.

1. CLASSES LIST grouped by batch and term, filters for department, class type
and teacher; counts of sections, students and sessions held.
2. CREATE CLASS: pick course, batch, term, class type; title auto-suggested
as "<Year> — <Course> — <Type>" (editable); default duration; venue. Assign
one or more teachers (faculty of any department) and mark one as Lead
(class_teachers).
3. On create, in one transaction: create a default section named "All"
(is_default = true) and, unless the admin unticks the option, enrol every
Active student of that batch into it with effective_from = term start date or
today, whichever is later.
4. ENROLMENT TAB on class detail: add students (search, select from batch, or
paste / upload a list of reg numbers, including repeaters from other
batches), end an enrolment with a date and reason. Enforce one open enrolment
per student per class (return a clear error if violated).
5. When a student's status changes to Freeze, Struck off, Left or Graduated,
end all their open enrolments on the effective date automatically.
6. COPY FROM PREVIOUS TERM: choose source term and target term/batch, preview
the classes, copy with teachers (and optionally sections).
7. Excel import for classes using the template: course_code, class_type,
title, teacher_codes (lead first), default_duration_min.
8. Class detail page: overview, teachers, sections (only "All" for now),
enrolled students, sessions history (empty for now).
9. Setup checklist: add "classes without teachers" and "classes without
students".
```

Check: create "1st Year — Anatomy — Lecture", "1st Year — Anatomy — Practical" and "1st Year — Physiology — Lecture"; each shows section All with the whole batch. Try enrolling one student twice and confirm the error.

Prompt 6 — Sections

Goal: classes can be split, each section has its own students and teachers.

```
[Paste the Context block here]

Build the Section Manager on the class detail page.

1. Create, rename, archive sections; set capacity, venue and section teachers
(section_teachers). Section teachers can mark only that section; class
teachers can mark any section of the class.
2. AUTO-SPLIT wizard (applies to students currently in the default section):
by roll-number ranges (admin types ranges), into N equal groups by roll
order, alphabetically by name, or copy the membership of another class. Show
a preview table (section -> roll range -> count) before applying.
3. Applying a split or moving a student between sections ends the old
enrolment (effective_to = day before the move date) and opens a new one in
the target section on the move date, in one transaction. Never update or
delete past attendance_records.
4. Drag-and-drop between section columns on desktop; checkbox select + "Move
to" on mobile. Show capacity warnings.
5. The default "All" section is archived automatically when it becomes empty
after a split, and restored if the admin merges sections back.
6. Every student enrolled in a class must be in exactly one section: show a
red banner on the class when anyone is unassigned and add it to the Setup
checklist.
7. Audit every split and move with the list of students.
```

Check: split Anatomy Practical into 4 groups by roll range; give Group 3 its own teacher; move one student from Group 1 to Group 2 and confirm their enrolment history shows both.

Prompt 6A — Groups and parallel group teaching

Goal: each MBBS year class is divided into groups once, those groups drive the practical and SGD sections, and different groups can be taught different subjects at the same time.

```
[Paste the Context block here]

Build the Groups module and the Teaching Slot planner (Campus Admin only; HOD
and teachers read-only where relevant).

1. GROUP SETS per year class (batch + term): name, purpose (teaching or
clinical). Groups inside a set with name, code and colour.
2. FILL WIZARD: by roll-number ranges, N equal groups by roll order,
alphabetical, Excel list (reg_no, group_code), or drag-and-drop. Every Active
student of the year class must be in exactly one group per set
(student_group_members, one open membership per set). Show an "Unassigned"
column and add it to the Setup checklist.
3. MOVING A STUDENT between groups takes a move date: close the old
membership, open the new one, and in the same transaction move the student's
enrolment in every section linked to those groups (end old enrolment the day
before, open new one on the move date). Past attendance never changes.
4. SECTION MANAGER: add "Split by group set" which creates one section per
group of a chosen set, sets sections.student_group_id, and enrols the group
members (enrollments.source = group_link). Linked sections show the group
colour. A class may keep "All" for lectures and use group sections for
practicals.
5. TEACHING SLOT PLANNER: week grid with time rows and one column per group
of a set. Creating a slot: date, start and end time, then for each group
choose class (section auto-picked as that class's section linked to the
group), teacher (staff in teaching category assigned to that class or
section), venue. Options: repeat weekly until a date; rotate groups through
the chosen classes each week.
6. CLASH CHECKS on save (and on every session create): no student in two
overlapping live sessions in any section; no teacher conducting two
overlapping sessions; no venue double-booked. Refuse with a message naming
the clashing session.
7. Saving a slot creates one planned (draft, empty-roster) session per group
with block/module tags from getCurriculumTag; the roster snapshot is taken
when the teacher opens it to mark.
8. SLOT STATUS VIEW for the admin: each slot as a row with a chip per group:
not started / draft / submitted / cancelled.
```

Check: create "Teaching groups A–F" for 1st Year; split Anatomy Practical, Physiology Practical and Biochemistry SGD by that set; plan Monday 09:00 with A, B, C in three different subjects; try to put Group A in two places at 09:00 and confirm it is refused; move one student from B to D and confirm their linked sections changed only from the move date.

Prompt 7 — Taking attendance (the first usable day)

Goal: a teacher signs in on a phone, picks a section and submits attendance in under a minute.

```
[Paste the Context block here]

Build the teacher attendance flow. It must be fast on a mid-range Android
phone over 4G.

1. MY CLASSES (teacher home): today's date, cards for every section the
teacher can mark (via class_teachers, section_teachers or as an assigned
substitute), each card showing class title, section, student count, last
session date and topic, and a large "Take attendance" button. Search across
their sections.
2. START SESSION (POST /sessions): date defaults to today (editable back to
backdate_days), start time now rounded to 5 minutes, end time from default
duration, session type from class, Topic required (suggest the last 5
topics), substitute toggle, default status choice (All present / All absent).
Server, in one transaction: verify rights and date limit, warn on calendar
holiday/closure (allow with reason), insert the session (overlap -> 409
showing the clashing session), insert one attendance_records row per student
whose enrolment in that section covers the date, return the roster sorted by
roll number.
3. MARKING SCREEN: sticky header (class, section, date, topic), bulk bar (All
present, All absent, Reset), list view with roll number (large), name and
small photo; tap cycles P > A > L > E; long-press opens a sheet for M
(medical) and a remark (remark required for E and M); compact grid view
toggle showing only roll numbers; search box; sticky footer with live
counters (P, A, L, E, M, total) and Save draft / Submit.
4. RELIABILITY: autosave every change to local storage; if the network drops,
show "Saved on this phone, will send when online" and retry automatically;
PUT /sessions/:id/records sends the session version and returns 409 if
someone else saved first. Submit sends an idempotency key so double taps
never create duplicates.
5. SUBMIT: confirmation dialog with counts and the list of absent roll
numbers; on success state = submitted, submitted_at set, audit entry written;
show the read view with "Editable for X h Y min" until lock.
6. DESKTOP: keyboard shortcuts (arrow keys move, P/A/L/E set status,
Ctrl+Enter submit).
7. ADMIN can start a session for any section and choose the conducting
teacher.
Seed 3 weeks of realistic sample sessions for one test section only if I ask;
do not seed fake data into real sections.
```

Check: on your phone, sign in as a teacher, mark a 50-student section, switch on airplane mode mid-way, switch it off, submit. Confirm one session only, correct counts, and that a second section of another teacher is not visible.

Prompt 8 — Lock, cancel, corrections and combined sessions

Goal: attendance cannot be quietly changed after the lock window, and every change after it is approved and recorded.

```
[Paste the Context block here]

Add the attendance control rules.

1. LOCK: a session is locked when state = locked, or when state = submitted
and now() is past session end + campus lock_hours. Enforce on every write in
the server. Add a scheduled job (or lazy check on read) that sets state =
locked and locked_at. Show the lock countdown on the session.
2. CANCEL: teacher (before lock) or admin can cancel with a required reason;
cancelled sessions are excluded from all calculations and shown struck
through in history.
3. CORRECTION REQUESTS: on a locked session the teacher taps "Request
correction", selects students, chooses new statuses, enters a reason. Queue
page for Campus Admin (and HOD of the class's department when
campus_settings.hod_can_approve) with filters and a badge count. Approve
applies all changes in one transaction and writes audit entries per student;
reject requires a comment. The teacher sees the outcome on the session.
4. ADMIN EDIT: Campus Admin can edit a locked session directly with a
mandatory reason; audited.
5. SUBSTITUTE: admin or HOD can assign a substitute teacher to a specific
date and section; the substitute sees it in My Classes for that date only.
6. COMBINED SESSION: a teacher selects two or more of their sections taught
together; one merged roster (grouped by section) is marked; the server
creates one session per section with a shared combined_group_id in one
transaction.
7. CAMPUS CLOSURE: adding a calendar event of type closure offers to cancel
all sessions on those dates with one reason.
8. SESSION DETAIL page: header, records, state timeline (created, submitted,
locked, corrections) built from audit_logs.
```

Check: set lock window to 1 hour for testing; submit a session, wait, confirm the teacher can no longer edit; request a correction, approve it as admin, and see it in the audit trail. Reset the lock window afterwards.

Prompt 8A — Clinical rotations and hospital attendance

Goal: clinical groups rotate through hospital departments on a plan, and supervisors mark each group's attendance in the ward, OPD or theatre.

```
[Paste the Context block here]

Build the Clinical Rotations module (Campus Admin plans; clinical supervisors
mark; HOD and Principal read-only).

1. CLINICAL CLASSES: for a clinical year class, a helper creates one class of
type clinical per selected hospital department ("<Year> — <Department> —
Clinical") with a section per clinical group, linked to the group.
2. ROTATION PLANNER: grid of clinical groups (rows) × weeks (columns),
coloured by department. Create a rotation: group, department, start and end
date, lead supervisor and other supervisors (active staff in the clinical
category of that department or its units). Saving enrols the group's members
into that department's clinical section with enrolment dates equal to the
rotation dates (enrollments.source = rotation). Refuse overlapping rotations
for one group (database EXCLUDE constraint) and show a friendly message. Copy
a plan to the next group with an offset to build a full cycle quickly.
3. ROTATION BOARD: this week's postings per department, for admin, Principal
and department heads (heads read-only, own department).
4. MY ROTATIONS for clinical supervisors: groups currently posted to them;
Take attendance asks for clinical sub-type (ward round, bedside teaching,
OPD, OT, emergency duty, case presentation/CPC, skills lab), hospital
location from the department's list, topic; then the same marking screen,
lock and correction rules as college classes. Sessions store rotation_id and
location_id.
5. UNIT SPLIT: optionally split a group posted to a department into sub-
groups per unit, each with its own supervisor.
6. PAPER FALLBACK: Campus Admin can enter a session from a paper sheet with
entered_from_paper = true and a photo of the sheet.
7. ROTATION COMPLETION REPORT: per rotation, each student's posting
attendance (held, attended, %) against campus clinical_threshold_pct, flags
for repeat posting, PDF/Excel export.
8. Emergency duty sessions count toward clinical % or are report-only
according to campus_settings.count_emergency_duty.
```

Check: create clinical groups 1–4 for the 4th Year (or a test batch), plan Group 1 in Medicine then Surgery, sign in as the Medicine supervisor and mark a ward round in Medical Ward 1; confirm Group 1 appears for the Surgery supervisor only from the Surgery start date.

Prompt 9 — Calculation engine and core reports

Goal: percentages are correct under every campus setting, and the three reports admins use daily exist.

```
[Paste the Context block here]

Build the attendance calculation engine and core reports.

1. calculateAttendance(counts, settings) in shared code, the ONLY place
attendance maths happens. Input: held, present, late, absent, excused,
medical, minutes_held, minutes_attended. Rules: late counts as present, or
every late_to_absent_n lates converts one to absent; excused and medical are
excluded from the denominator or counted as present per excused_rule; basis =
sessions or contact hours. Output: attended, denominator, pct (1 decimal),
band ok / at_risk (pct < warning_pct) / shortage (pct < threshold_pct).
Division by zero returns null and band "no_sessions".
2. Unit tests for calculateAttendance covering at least these cases from the
spec: P30 L3 A3 M2 held 38 -> 91.7% (late present, medical excluded); same
with 3 lates = 1 absence -> 88.9%; medical counted present -> 92.1%; zero
held; all excused. Run the tests and show me the results.
3. Six levels: per class (v_student_class_attendance), per course /
discipline (all of the student's classes of that course), per module and per
block (sessions grouped by their block_id / module_id tags), per clinical
rotation (sessions with that rotation_id, using clinical_threshold_pct), and
per term. Build SQL queries for each, filtered by campus_id, with group
filters (show a group's students only).
4. SECTION REGISTER: pick class, section, month; grid of students (rows, roll
order) x sessions (columns, date + time, topic on hover) with P/A/L/E/M
chips, cancelled sessions greyed, totals and % per student at the end; sticky
first column and header.
5. STUDENT ATTENDANCE CARD: per student and term, a table of every class
(held, attended, %, band) plus per course and overall; visible from the
student profile.
6. SHORTAGE LIST: filters for batch, course or class, level (class / course /
term), threshold override; lists students at Shortage and At risk with %,
held, attended; sorted by % ascending.
7. Exports for all three: A4 PDF with ABWA Knowledge Village and campus logo,
report title, filters, generated date and user; Excel with the same columns.
Audit every export.
```

Check: pick 3 students and calculate their % by hand from the section register; the app must match exactly. Change the late rule and confirm every report changes the same way.

Prompt 10 — Dashboards and remaining reports

```
[Paste the Context block here]

Build dashboards and the remaining reports.

1. CAMPUS DASHBOARD (Campus Admin, Principal; HOD sees own department): KPI cards for today (sessions held, campus present %, sections not marked yet, pending corrections, students in Shortage), a 30-day line chart of daily present %, a bar chart of present % by batch, a table of today's unmarked sections with teacher and a "remind" copy-text button.
2. VILLAGE DASHBOARD (Super Admin): one card per campus with today's present %, active students, unmarked sections; a 30-day chart comparing campuses.
3. TEACHER HOME additions: their sessions this week, any sessions still in draft, correction outcomes.
4. REPORTS page listing: Daily campus summary, Section register, Student card, Shortage list, Class log / teaching record (sessions, topics, conducted by, for a class / teacher / date range), Teacher activity (sessions conducted, drafts never submitted, sessions submitted more than 24 h late), Correction log, Village overview. Each with filters, loading and empty states, PDF and Excel export.
5. Use the summary queries; never load all records into the browser. Paginate tables over 200 rows.
```

Check: dashboards load in under 3 seconds for the pilot batch; numbers match the reports.

Prompt 11 — Hardening, security review and backups

```
[Paste the Context block here]

Harden ABWA CMS before real use.

1. SECURITY REVIEW: go through every API route and list: required role,
campus filter present (yes/no), permission check used. Fix any route missing
either. Add automated tests proving: an AMC admin cannot read or write any
ANC row; a teacher cannot read or mark a section they are not assigned to
(including by guessing IDs); a locked session cannot be changed by a teacher;
a disabled user is refused.
2. ROW-LEVEL SECURITY: enable PostgreSQL RLS on campus-owned tables with a
policy using a per-request setting (SET LOCAL app.campus_ids) set by the
server in each transaction, so a missed filter returns nothing. Super Admin
requests set all campuses.
3. Make audit_logs append-only for the application's database user.
4. BACKUPS: add a scheduled nightly export of the database to a Cloud Storage
bucket with 30-day retention, and a Super Admin "Download full backup
(Excel)" action. Tell me exactly what I must configure in Google Cloud for
this.
5. RATE LIMITS on sign-in and exports; upload size limits; helmet-style
security headers; CORS restricted to the app domain.
6. ERRORS: friendly error pages, a global error boundary, server error
logging with request id; a "waking up the database" loader for the first slow
request.
7. PERFORMANCE: check query plans for the section register, shortage list and
dashboard queries; add missing indexes via a migration.
8. ACCESSIBILITY AND MOBILE: test every screen at 360 px; tap targets 44 px;
status shown by letter and colour.
```

Check: run the generated tests; read the security review list yourself; confirm a backup file appears in the bucket the next morning.

Prompt 12 — Pilot data, help page and publish

```
[Paste the Context block here]

Prepare ABWA CMS for the Medical College pilot.

1. A "Go-live" page for Campus Admin that runs the Setup checklist and blocks
nothing but lists every gap with links.
2. A Help page with short guides and screenshots placeholders: "Taking
attendance" (teacher), "Requesting a correction" (teacher), "Splitting a
class into sections" (admin), "Reading the shortage list" (admin).
3. A feedback button on every page that saves {user, page, message} to a
feedback table for the Super Admin.
4. Remove any demo data from real campuses; keep the ABWA Nursing College
campus only as an empty test campus or archive it.
5. Review the app for anything that still uses placeholder text, then tell me
the publish checklist (environment secrets, Firebase authorised domains,
database size, backups) before I click Publish.
```

Then: click **Publish** in AI Studio, add the published domain to Firebase authorised domains if asked, sign in from a phone, and run the go-live checklist in section 13 with the pilot teachers.

13. Testing, go-live and troubleshooting

Go live with one batch and a handful of teachers for two weeks, keep the paper register in parallel, and compare the two before rolling out to the whole college.

13.1 Acceptance tests

- [ ] Medical College admin cannot see or open any Nursing College record, by menu or by URL.
- [ ] Teacher sees only assigned sections; cannot open another teacher's section by changing the ID in the URL.
- [ ] Undivided class works through its single "All" section.
- [ ] Anatomy Practical split into 4 groups keeps 4 separate registers and percentages.
- [ ] Student moved between groups keeps old marks; class % covers both periods.
- [ ] 150 students marked and submitted on a phone in under 60 seconds.
- [ ] Marks survive airplane mode and are sent once when the network returns; no duplicate sessions.
- [ ] Overlapping session for the same section is refused with a clear message.

- [ ] Teacher cannot edit after the lock window; correction approve/reject works and is audited.
- [ ] Cancelled sessions are excluded; excused/medical follow the campus rule.
- [ ] Percentages match a hand calculation for 3 students under 2 different late rules.
- [ ] Shortage list shows exactly the students below threshold.
- [ ] Section register prints on one A4 page per 30 students with logos.
- [ ] Excel imports reject duplicates and list row errors.
- [ ] Disabled user is refused immediately.
- [ ] Nightly backup file exists and opens.

13.2 Go-live plan

1. **Week 0:** load real data for one batch; train 3–5 pilot teachers (15 minutes each, on their own phones).
2. **Weeks 1–2:** teachers mark in the app and on paper; the Campus Admin checks the dashboard daily for unmarked sections.
3. **End of week 2:** compare app percentages with paper for 10 students; fix gaps; collect feedback from the feedback table.
4. **Week 3 onwards:** all batches of the Medical College, then stop paper registers.
5. **Next campus:** create ABWA Nursing College's admin and repeat from section 6.1, step 4.

13.3 Troubleshooting prompts

When something breaks, describe what you did, what you expected and the exact error, and ask for a fix to that one thing. These ready-made prompts cover the common failures in AI-built apps.

Problem	Prompt to paste
Data disappears or the agent wants to recreate tables	"Stop. Do not drop or recreate any table. Show me the current schema and the last migration, then make this change only with a new additive migration."
A screen shows other campus data	"Campus isolation bug on <page>. List the API route and SQL it uses, show where campus_id is filtered, fix it, and add a test proving an AMC user cannot see ANC data there."

Problem	Prompt to paste
Percentages look wrong	"For student <reg no> in <class>, show the raw counts from v_student_class_attendance, the campus settings, and the step-by-step output of calculateAttendance. Then fix the discrepancy and add a unit test for this case."
Teacher can't see their section	"Teacher <email> cannot see <section>. Show their role_assignments, faculty row, class_teachers and section_teachers rows, and the exact query My Classes runs. Fix the cause, not the symptom."
Duplicate sessions appear	"Duplicate sessions were created for <section> on <date>. Show how idempotency_key and the overlap constraint are used on POST /sessions and submit, fix the gap, and add a test."
App feels slow	"The <page> takes <n> seconds. Show the SQL it runs with EXPLAIN ANALYZE, add the right indexes in a migration, and avoid loading records into the browser."
The agent changed unrelated screens	"Revert changes outside <module>. Only modify files under <module> for this request and list every file you touch before changing it."
Sign-in fails after Publish	"Sign-in works in preview but not on the published URL. Check Firebase authorised domains, environment secrets and the API base URL, and tell me exactly what to change in which console."

14. Future modules and decisions to confirm

Every future module reuses the core's campuses, people, classes and sections, and registers itself in the module registry.

Phase	Module	Plugs into
2	Timetable: weekly slots per section, auto-creates expected sessions, flags unmarked ones	sections, faculty, sessions
2	Student portal: own attendance, shortage warnings	students, attendanceSummaries
2	Leave management: student applies, admin approves, auto-marks E/M	students, sessions
2	Notifications: SMS/WhatsApp/email to students and guardians on absence or shortage	attendanceSummaries, students
3	Examinations: class tests, ward tests, professional exam eligibility (blocks students in Shortage)	courses, classes, attendanceSummaries
3	Faculty attendance and leave	faculty, sessions
3	Fee management	students, batches
4	LMS: lecture notes, assignments per section	classes, sections
4	Hostel, transport, library	students, campuses
4	Parent portal and mobile app	students, all modules

Decisions to confirm before building

- [ ] Attendance threshold for MBBS (default 75%) and warning band (default 80%), checked against current PMDC and affiliating university rules.
- [ ] Is threshold applied per class, per course (lecture + practical combined), or both?
- [ ] Calculation by session count or by contact hours.
- [ ] Late rule: counts as present, or 3 lates = 1 absence?
- [ ] Excused/medical leave: excluded from total, or counted as present?
- [ ] Lock window (default 24 h) and back-dating limit (default 7 days).
- [ ] Who approves corrections: Campus Admin only, or HOD as well?
- [ ] Registration number and roll number formats for each campus.
- [ ] Whether the Medical College follows subject-wise or integrated modular classes for each year (affects how courses are set up).
- [ ] Do students get logins in Phase 1, or only from Phase 2?